

LA PROGRAMMATION EN LANGAGE C

INTRODUCTION

MIP/ S3

Pr S.EL FILALI
sanaa.elfilali@univh2c.ma

Objectifs du cours

une liste des compétences à acquérir

- comprendre la syntaxe C,
- déclarer des variables,
- utiliser les opérateurs basiques.

SOMMAIRE

- Algorithme
- Programmation
- Programme
- Historique du langage C
- Le choix du C
- Langages et systems écrits en C
- Les étapes de programmation
- Les étapes de compilation
- L'Editeur
- Le compilateur
- L'Edition de liens
- L'Anatomie d'un programme en C
- La Structure d'un programme
- Exercice
- Questions de cours

L'ALGORITHME ?

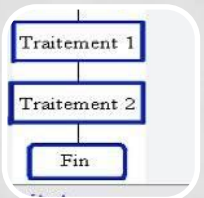
Un algorithme :



Mot dérivé du nom du mathématicien al_Khwarizmi qui a vécu au 9^{ème} siècle, était membre d'une académie des sciences à Bagdad .



Un algorithme décrit un traitement sur un ensemble fini de données, il est constitué d'un ensemble fini d'actions composées d'opérations ou actions élémentaires.



Nécessite un langage clair (compréhension) structuré (enchaînements d'opérations) non ambiguë, universel (indépendants du langage de programmation choisi)

LA PROGRAMMATION ?

la **programmation** informatique :



communication avec la machine (pour dire à une machine ce qu'elle doit faire, la guider),



c'est ce qui permet l'écriture des programmes pour développer des logiciels, ou une page web,



Un programme est une liste d'instructions écrites pour résoudre un problème, ou pour effectuer une action.

PROGRAMME ?

Liste d'*instructions*, écrites (par vous !)



dans un ou plusieurs *fichiers*



destinées à être *exécutées* (à votre demande)



par l'ordinateur

HISTORIQUE DU LANGAGE C

Le langage C a été créé

- Dennis Ritchie (NB) amélioration du B
- Ken Thompson (langage B)
- Brian Kernighan (a aidé à le populariser)
- au début des années 70 (1972)



Le langage C possède Trois ancêtres

- CPL Combined Programming Language
- BCPL Based Combined Programming Language
- B

Pourquoi?

- Le **langage C** a été inventé pour écrire le système d'exploitation UNIX,
- Ainsi le noyau de grands systèmes d'exploitation comme Windows et Linux sont développés en grande partie en **C**.
- afin d'éviter autant que possible l'utilisation de l'assembleur dans l'écriture du système UNIX

L'avantage du C sur l'assembleur :

- plus simples à écrire
- facilement portables sur d'autres architectures.

LE CHOIX DU C

Pourquoi le C ?

Le C est un langage

de haut
niveau,

capacités
d'accéder

permet de
créer

possède peu
d'instructions
.

programmation
structurée.

une grande
popularité.

amène au
C++

des commandes
de bas niveau.

des exécutables
très rapides
(compilé),

principal
langage
orienté-objet

LES LANGUAGES ET SYSTEMS ÉCRITS EN C

Les languages et systems

JVM

Java Virtual
Machine

Harduino

UNIX

PHP

C++

...

un programme
java est
exécuté dans
une Java
Virtual Machine

Carte
électronique
à source
ouverte

système
d'exploitatio
n multitâche
et multi-
utilisateur),

LES ÉTAPES DE PROGRAMMATION

Une approche à quatre étapes :

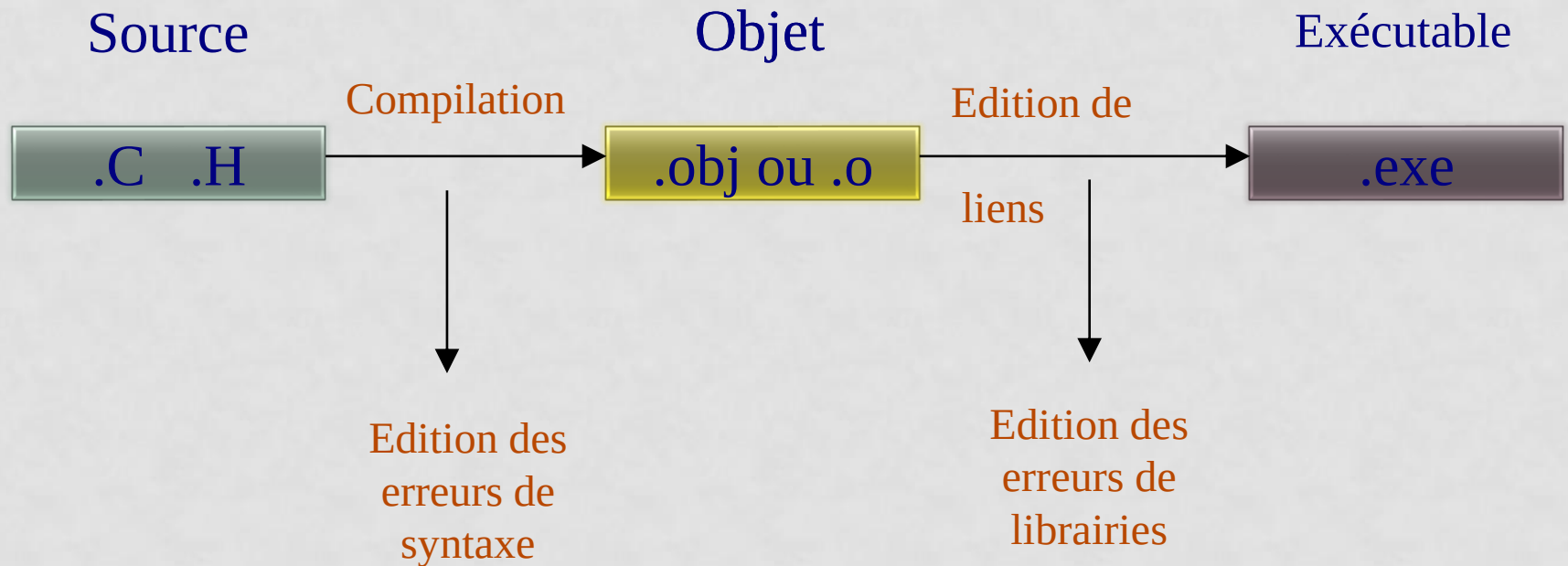
Éditer,

Compiler,

Édition
de liens
("Link"),

Exécution
("Run").

LES ÉTAPES DE COMPILE



L'ÉDITEUR

On utilise un éditeur de texte pour écrire le programme.

L'éditeur nous permet de conserver notre code

sur un fichier,

sur un disque.

LE COMPILATEUR

Le compilateur

traduit le code de haut-niveau (C)

en un langage de bas-niveau (langage machine)

Le fichier où est stocké le code de bas niveau

est appelé fichier objet

L'EDITION DE LIENS

L'éditeur de liens ("linker")

réunit

les codes objets et les ressources existantes (bibliothèques).

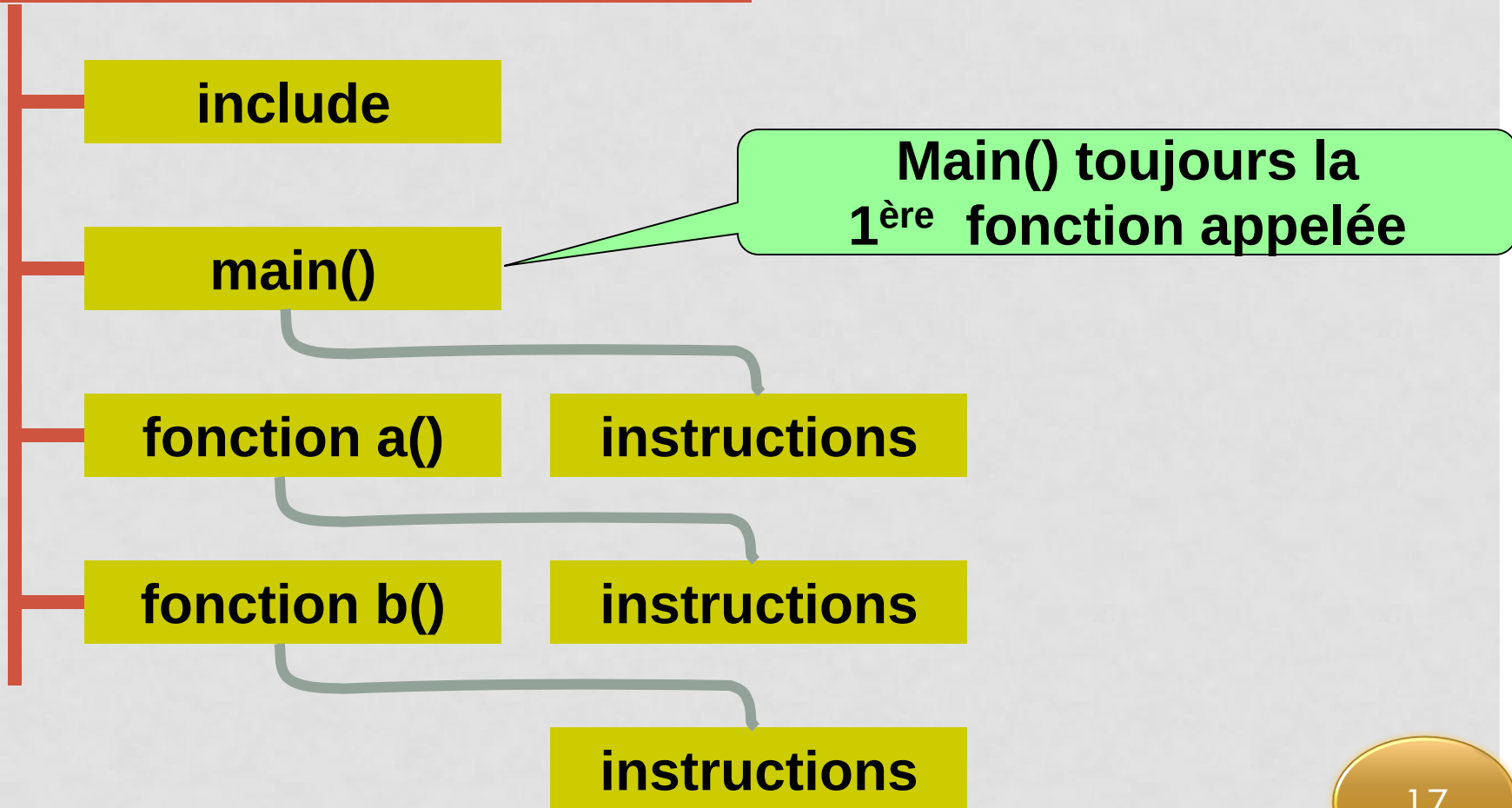
produits par le compilateur

Le résultat de l'éditeur de liens

un fichier exécutable.

ANATOMIE

Programme typique en C



STRUCTURE D'UN PROGRAMME EN C

Structure d'un programme C

```
#include <stdio.h>
#define DEBUT -10
#define FIN 10
#define MSG "Programme de démonstration\n"
int fonc1(int x);
int fonc2(int x);
```

Directives du préprocesseur :
accès avant la compilation

Déclaration des fonctions

```
void main()
```

```
{
    int i;
```

```
    i = 0 ;
    fonc1(i) ;
    fonc2(i) ;
```

```
}
```

```
int fonc1(int x) {
    return x;
}
```

```
int fonc2(int x) {
    return (x * x);
}
```

/ début du bloc de la fonction main */*
/ définition des variables locales */*

/ fin du bloc de la fonction main */*

Programme
principal

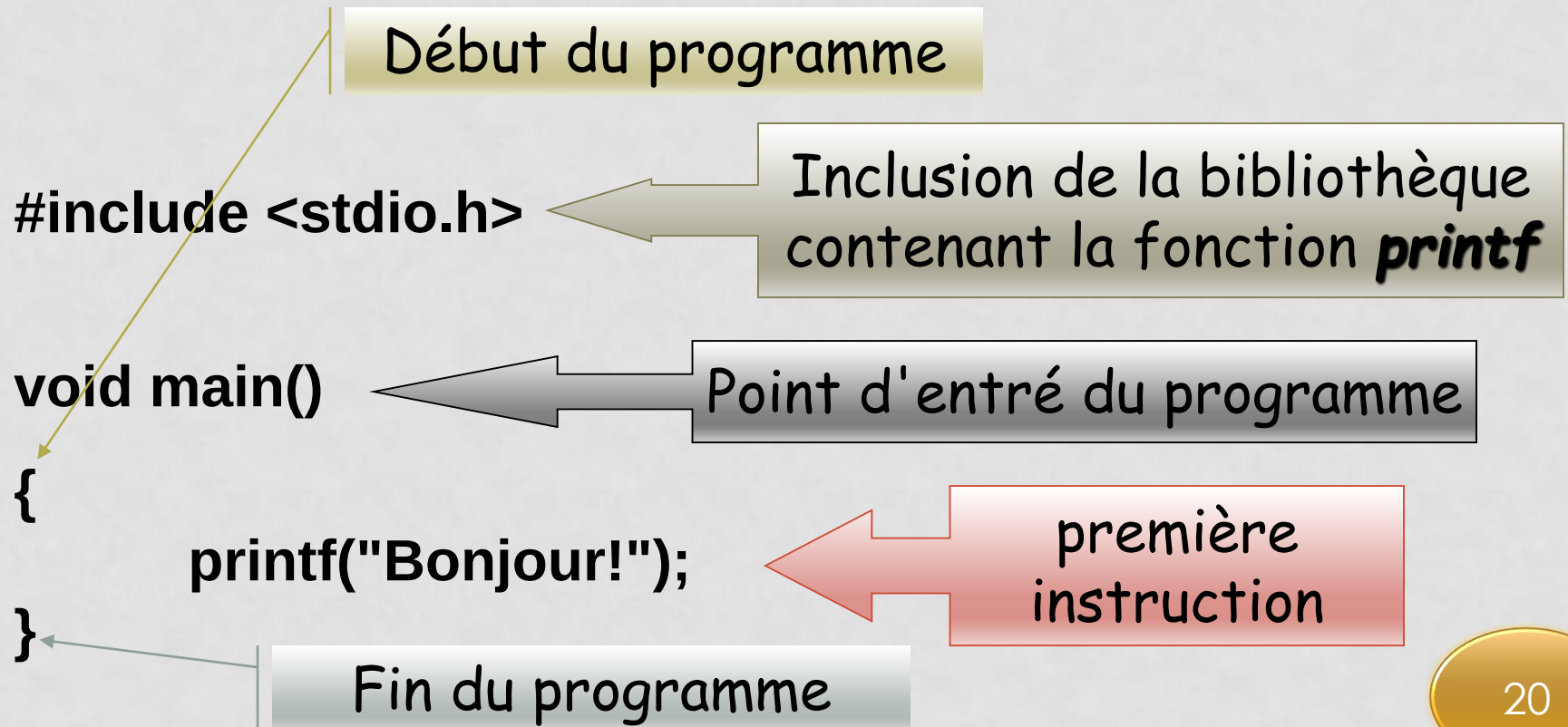
Définitions des
fonctions

EXERCICE1.1

- z Créer un fichier Hello.c
- z Ecrire dans ce fichier un programme C permettant d'afficher « **Bonjour** » à l'écran
- z Compiler
- z Exécuter

SOLUTION EXERCICE1.1

z Premier Programme en C



QUESTIONS RÉPONSES

- Une petite histoire sur Le langage C
- Définition :
 1. un algorithme,
 2. un programme
 3. un langage de programmation
 4. un assembleur
 5. unix
- Pourquoi le langage C ?
- Les langages de programmations écrits et développés en C ?
- Quelles sont Les étapes de programmation ?
- C'est quoi un Fichier source ?
- C'est quoi un Fichier exécutable ?
- C'est quoi un Fichier objet ?
- Qu'est ce qu'un Script ?
- C'est quoi un Langage interprété ?
- C'est quoi un Langage compilé ?
- C'est quoi un Bytecode ?
- Un langage de haut niveau ?
- Un langage de bas niveau ?

LA PROGRAMMATION EN LANGAGE C

CHAPITRE 2 : LES VARIABLES

MIP/ S3

Pr S.ELFILALI

sanaa.elfilali@etu.univh2c.ma

Une variable
les variables typées
Les caractéristiques d'une variable
Les types de variable
Les types numériques entiers
Les types rationnels
Déclaration d'une variable
Initialisation
Déclaration d'une constante
Identificateurs
Incrémentatation / décrementation
Préfixe et postfixe
Conversion des types

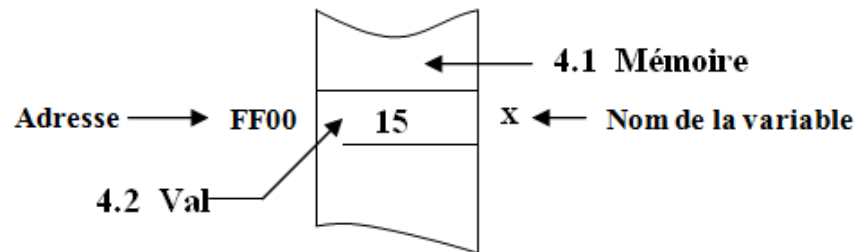
UNE VARIABLE ?

Une variable est un objet

Repéré par son
nom

Pouvant
contenir des
données

Pourront être
modifiées lors de
l'exécution du
programme.



LES VARIABLES TYPÉES

Les variables en langage C sont typées,

les données possèdent un type

elles sont stockées dans la mémoire

occupent un nombre d'octets

dépendant du type de donnée stockée.

LES CARACTÉRISTIQUES D'UNE VARIABLE

Nom

- Unique pour chaque variable
- Commence toujours par une lettre
- Différenciation minuscule-majuscule

type

- Conditionne le format de la variable en mémoire
- Peut être soit un type standard ou un type utilisateur

Valeur

- Peut évoluer pendant l'exécution
- initialisation grâce à l'opérateur d'affectation

Adresse

LES TYPES DE DONNÉES

Types de variable

Char	→	caractères
Int	→	entiers
short [int]	→	entiers courts
long [int]	→	entiers longs
Float	→	nombres décimaux
Double	→	nombres décimaux de précision supérieure
long double	→	nombres décimaux encore plus précis
unsigned int	→	entier non signé

TYPES NUMÉRIQUES ENTIERS

Type de donnée	Taille (en octets)	Plage de valeurs acceptée
char	1	-128 à 127
unsigned char	1	0 à 255
short int	2	-32768 à 32767
unsigned short int	2	0 à 65535
int	2	-32768 à 32767
unsigned int	2	0 à 65535
long int	4	-2 147 483 648 à 2 147 483 647

LES TYPES RATIONNELS

Type de donnée	Taille (en octets)	Plage de valeurs acceptée
float	4	-3.4×10^{-38} à 3.4×10^{38}
double	8	1.7×10^{-308} à 1.7×10^{308}
long double	10	3.4×10^{-4932} à 3.4×10^{4932}

DÉCLARATION D'UNE VARIABLE

Les déclarations permettent de :

réserver de l'espace en mémoire
pour les variables.

```
Type  nom_de_la_variable [= valeur] ;
```


DÉCLARATION D'UNE VARIABLE

Type nom_de_la_variable [= valeur] ;

Exemple:

int nb;

float pi = 3.14;

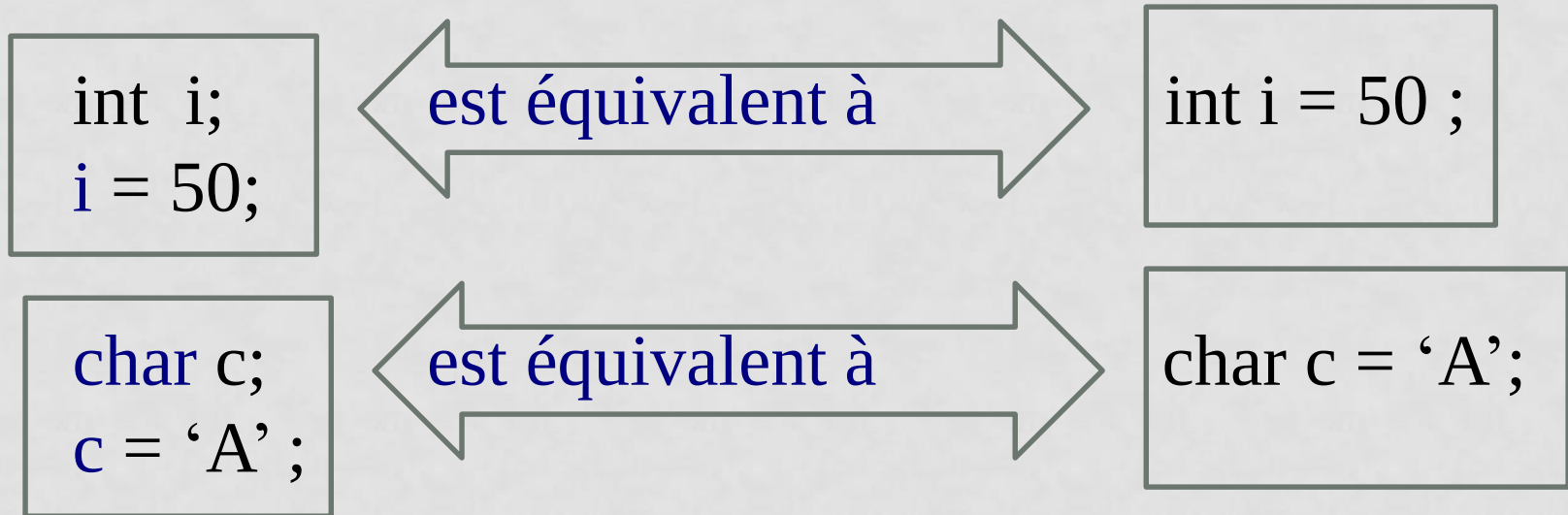
char c = 'a';

long i,j,k;

double r = 6.2879821365;

LES INITIALISATIONS

C permet l'initialisation des variables dans la zone de déclaration



LES DECLARATIONS DE CONSTANTES

1ere méthode :

définition d'un symbole à l'aide de la **directive** de compilation **#define**.



Le compilateur ne réserve pas de place en mémoire

Syntaxe : **#define** **identificateur** texte(valeur)

Exemple:

```
#define      PI  3.14159
void main()
{
    float perimetre, rayon = 8.7;
    perimetre = 2*rayon*PI;
    ....
}
```

LES DECLARATIONS DE CONSTANTES

2ème méthode :

déclaration d'une variable, dont la valeur sera constante pour tout le programme.

➤ **Le compilateur réserve de la place en mémoire**

Exemple

```
void main()
{
    const float PI = 3.14159;
    const int JOURS = 5;
    float perimetre, rayon = 8.7;
    perimetre = 2*rayon*PI;
    ....
    JOURS = 3;
    ....
}
```

Le compilateur réserve de la place en mémoire (ici 4 octets).

*/*ERREUR !*/ On ne peut changer la valeur d'une const.*

IDENTIFICATEURS

Les identificateurs nomment les objets C (fonctions, variables ...)

- **C'est une suite de lettres ou de chiffres.**
- **Le premier caractère est obligatoirement une lettre.**
- **Le caractère _ (souligné) est considéré comme une lettre.**

IDENTIFICATEURS

Le C distingue les minuscules des majuscules.

Exemples :

abc, Abc, ABC sont des identificateurs valides et tous différents.

Identificateurs valides :

xx	z2	somme_5	_position
Noms	surface	fin_de_fichier	VECTEUR

Identificateurs invalides :

5eme
x#z
no-commande
taux change

commence par un chiffre
caractère non autorisé (#)
caractère non autorisé (-)
caractère non autorisé (espace)

IDENTIFICATEURS

Un identificateur ne peut pas être un mot réservé du langage :

auto	double	int	struct
break	else	long	switch
case	enum	register	typedef
char	extern	return	union
const	float	short	unsigned
continue	for	signed	void
default	goto	sizeof	volatile
do	if	static	while

RQ : Les **mots réservés** du langage C doivent être écrits **en minuscules.**

INCRÉMENT ET DÉCREMENT

- C a deux opérateurs spéciaux pour incrémenter (ajouter 1) et décrémenter (retirer 1) des variables entières
 - ++ **increment** : $i++$ ou $++i$ est équivalent à $i += 1$ ou $i = i + 1$
 - -- **decrement**
- Ces opérateurs peuvent être :
 - préfixés (avant la variable)
 - postfixés (après)

“i” vaudra 6

“j” vaudra 3

“i” vaudra 7

```
int  i = 5, j = 4;  
    i++;  
    --j;  
    ++i;
```


PRÉFIXE ET POSTFIXE

```
#include <stdio.h>
```

```
int main(void)  
{
```

```
    int    i, j = 5;
```

```
    i = ++j;  
    printf("i=%d, j=%d\n", i, j);
```

```
    j = 5;  
    i = j++;
```

```
    printf("i=%d, j=%d\n", i, j);
```

```
    return 0;
```

```
}
```

équivalent à:

1. j++;
2. i = j;

équivalent à:

1. i = j;
2. j++;

i=6, j=6
i=5, j=6

LES CONVERSIONS DE TYPES

Le langage C permet d'effectuer des opérations de conversion de type. On utilise pour cela l'opérateur de "cast" ().

```
#include <stdio.h>
#include <conio.h>
void main()
{
    int i=0x1234, j;
    char d,e;
    float r=89.67,s;
    j = (int)r;
    s = (float)i;
    d = (char)i;
    e = (char)r;
    printf("Conversion float -> int: %5.2f -> %d\n",r,j);
    printf("Conversion int -> float: %d -> %5.2f\n",i,s);
    printf("Conversion int -> char: %x -> %x\n",i,d);
    printf("Conversion float -> char: %5.2f -> %d\n",r,e);
    printf("Pour sortir frapper une touche ");
    getch();    // pas getchar
}
```

Conversion float -> int: 89.67 -> 89

Conversion int -> float: 4660 -> 4660.00

Conversion int -> char: 1234 -> 34

Conversion float -> char: 89.67 -> 89

Pour sortir frapper une touche

AFFICHAGE

La fonction printf()

- Sortie de nombres ou de texte à l'écran
- Printf une fonction de la bibliothèque `stdio.h`

AFFICHAGE D UN TEXTE

```
printf("BONJOUR");
```

pas de retour à la ligne du curseur après l'affichage,

```
printf("BONJOUR\n");
```

affichage du texte, puis retour à la ligne du curseur.

AFFICHAGE D'UNE VARIABLE

Fonction de la bibliothèque stdio.h

```
printf("%format",variable);
```

format :

d

- pour les décimaux,

x

- pour les hexadécimaux,

c

- pour un caractère, etc.

AFFICHAGE D'UNE VARIABLE

Fonction de la bibliothèque `stdio.h`

Format : `printf("%format",variable);`

Exemples :

```
printf("bonjour");
```

```
printf("la valeur est : %d ",i);
```

```
printf("les valeurs sont : %d et %d",i,j);
```

SPÉCIFICATEURS DE FORMAT POUR PRINTF()

SYMBOLE	TYPE	IMPRESSION COMME
%d ou %i	int	Entier relatif
%u	int	Entier naturel (unsigned)
%o	int	Entier exprimé en octal
%x	Int	Entier exprimé en hexadécimal
%c	int	caractère
%f	double	rationnel en notation décimale
%e	double	rationnel en notation scientifique
%s	char*	chaîne de caractères

AFFICHAGE MULTIPLE

```
printf("format1.... formatN",variable1,....,variableN);
```


A suivre!!

Exemple printf

Exemple 1:

```
Int q=100;  
Float p = 60.50;  
Char t = 'A';  
printf("quantite = %d \n ",q);  
printf(" Prix= %f\n ",p);  
printf(" Taille= %c \n ",t);  
printf(" Prix Total =%f \n ",q*p);
```

ou bien

```
printf("Quantite = %d\nPrix = %f\nTaille = %c\nPrix total = %f",q,p,t,q*p);
```

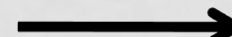
Imprime :

```
Quantite = 100  
Prix = 60.500000  
Taille = A  
Prix total = 6050,000000
```

Exemple printf

Exemple 2:

float val = 25.1234;

<code>printf("%f",val);</code>		<i>/* affiche : 25.123400 */</i>
<code>printf("%.0f",val);</code>		<i>/* affiche : 25 */</i>
<code>printf("%.1f",val);</code>		<i>/* affiche : 25.1 */</i>
<code>printf("%.2f",val);</code>		<i>/* affiche : 25.12 */</i>
<code>printf("%e",val);</code>		<i>/* affiche : 2.512340e+01 */</i>
<code>printf("%.3e",val);</code>		<i>/* affiche : 2.512e+01 */</i>

EXERCICE1.2

- ❑ Ecrire un programme C permettant de créer 2 variables
2 caractères initialisés à 65 et 'A'
- ❑ Afficher ces variables sous forme d'entiers et de caractères

SOLUTION EXERCICE 1.2

```
#include <stdio.h>
#include <conio.h>

void main()
{
    char u,v;
    u=65;
    v='A';
    printf("u = %d, %c",u,u);
    printf("v= %d, %c",v,v);
    getch();
}
```

LES OPÉRATEURS

Il existe 4 types d'opérateurs:

Arithmétiques

Logiques

Relationnels

Assignment.

OPÉRATEURS ARITHMÉTIQUES

+

- addition

-

- soustraction

*

- multiplication

/

- division (entière et rationnelle!)

%

- modulo (reste d'une div. entière)

EXERCICE 1.3

Ecrire un programme C déclarant 2 variables entières et affichant leur somme multipliée par 2.

SOLUTION EXERCICE 1.3

```
#include <stdio.h>
#include <conio.h>
void main ()
{
    int a,b,somme;
    a=10;
    b=50;
    somme=(a+b)*2;
    printf("résultat : %d\n",somme);
    printf("Appuyer sur une touche\n");
    getch();
}
```

OPÉRATEURS DE COMPARAISON

Ces opérateurs produisent une valeur booléenne vraie (**true**) ou fausse (**false**).

==

- égal à

!=

- différent de

<, <=, >, >=

- plus petit que, ...

OPÉRATEURS LOGIQUE

&& et logique (**and**)

|| ou logique (**or**)

! négation logique (**not**)

OPÉRATEURS LOGIQUE

Les résultats des opérations de comparaison et des opérateurs logiques sont du type int:

- la valeur **1** correspond à la valeur booléenne vrai
- la valeur **0** correspond à la valeur booléenne faux

OPÉRATEURS LOGIQUE

Les opérateurs logiques considèrent :

toute valeur **différente de zéro** comme vrai

Et toute **valeur zéro** comme faux.

OPÉRATEURS DE MANIPULATION DE BITS

Fonction	Symbole	Remarques
ET	&	$c = a \& b$; $\rightarrow$ si $a = 1001\ 0011$ et $b = 0101\ 0110$, $c = 0001\ 0010$
OU		$c = a b$; $\rightarrow$ si $a = 1001\ 0011$ et $b = 0101\ 0110$, $c = 1101\ 0111$
OU exclusif	^	$c = a \wedge b$; $\rightarrow$ si $a = 1001\ 0011$ et $b = 0101\ 0110$, $c = 1100\ 0101$
Décalage à droite	>>	$(a \gg 2)$ décale la valeur de a de 2 bits vers la droite
Décalage à gauche	<<	$(a \ll 2)$ décale la valeur de a de 2 bits vers la gauche
Complément à un	~	$c = \sim a \rightarrow$ si $a = 1001\ 0011$, $c = 0110\ 1100$

```
unsigned char u_c = 1;
```

0	0	0	0	0	0	0	1
---	---	---	---	---	---	---	---

```
u_c = u_c << 1;
```

0	0	0	0	0	0	1	0
---	---	---	---	---	---	---	---

```
u_c = u_c << 2;
```

0	0	0	0	1	0	0	0
---	---	---	---	---	---	---	---

```
u_c = u_c >> 3;
```

0	0	0	0	0	0	0	1
---	---	---	---	---	---	---	---

LES OPÉRATEURS D'AFFECTATION

En pratique, nous retrouvons souvent des affectations comme:

$$i = i + 2$$

En C, nous utiliserons plutôt la formulation plus compacte:

$$i += 2$$

L'opérateur `+=` est un *opérateur d'affectation*.

LES OPÉRATEURS D'AFFECTATION

Somme = somme + i;

Ici, la valeur de la variable **somme** est additionnée à **i** puis réassignée (ensuite !) à la même variable **somme**.

LES OPÉRATEURS D'AFFECTATION

Pour la plupart des expressions de la forme:

$$\text{expr1} = (\text{expr1}) \text{ op } (\text{expr2})$$

$$\text{expr1} \text{ op} = \text{expr2}$$

+=

ajouter à

-=

diminuer de

***=**

multiplier par

/=

diviser par

%=

modulo

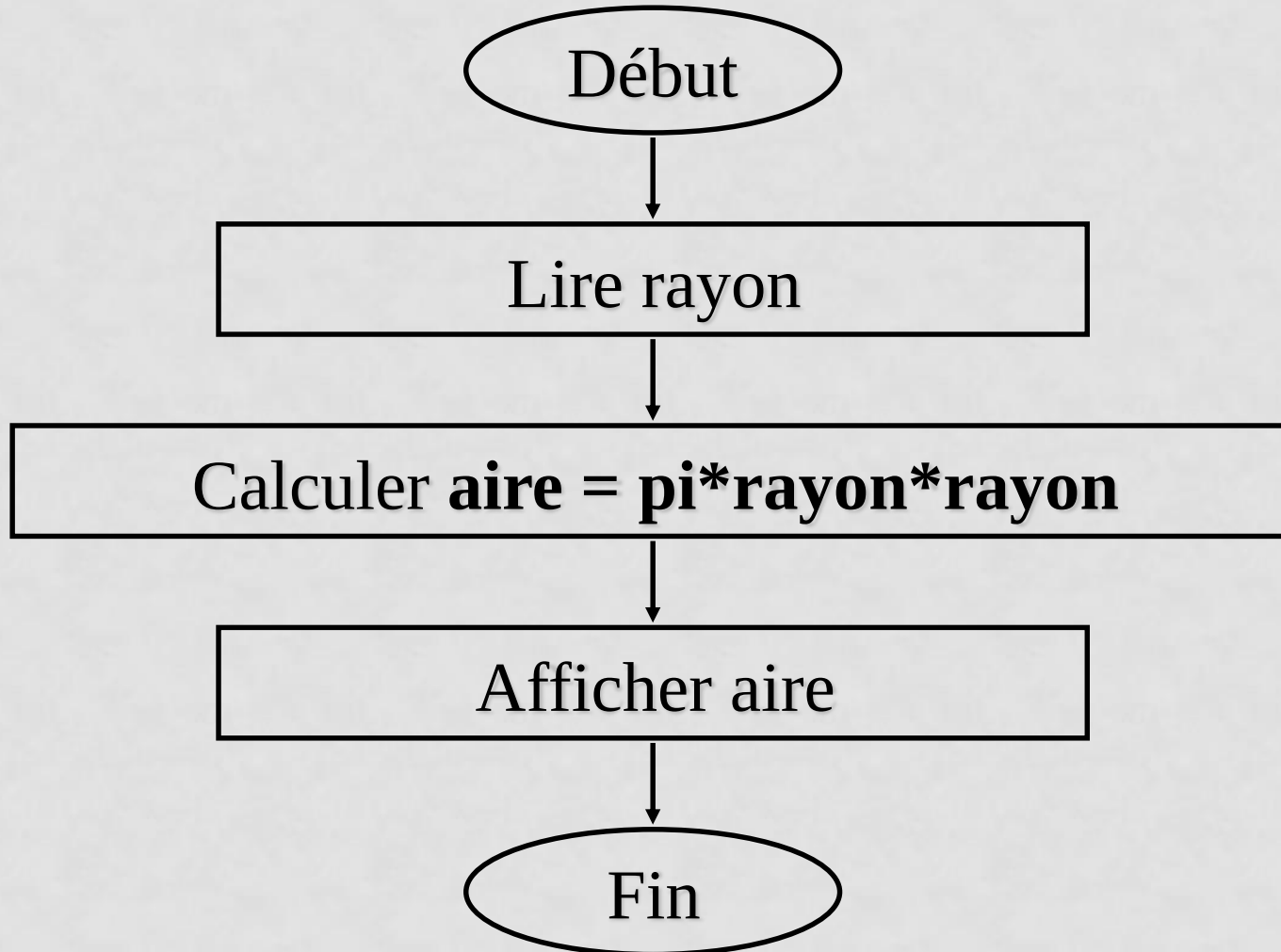
LES OPÉRATEURS ABRÉGÉS (OPÉRATEURS D'ASSIGNEMENT)

opérateur abrégé	Expression abrégée	Expression équivalente
+=	c += 3;	c = c + 3;
-=	c -= 3;	c = c - 3;
*=	c *= 3;	c = c * 3;
/=	c /= 3;	c = c / 3;
%=	c %= 3;	c = c % 3;

Priorité des opérateurs

N priorité	Type d'opérateur	Opérateurs	Associativité
15	Primaire	() [] . ->	g à d
14	Unaire	++ -- (nom Type) * & sizeof + - ! ~	g à d
13	Arithmétique	* / %	g à d
12	Arithmétique	+ -	g à d
11	De manipulation de bits	>> <<	g à d
10	De relation	> < >= <=	g à d
9	De relation	== !=	g à d
8	De manipulation de bits	&	g à d
7	De manipulation de bits	^	g à d
6	De manipulation de bits		g à d
5	Logique	&&	g à d
4	Logique		g à d
3	Conditionnel	?:	g à d
2	D'affectation	= += -= *= /= = ^=	g à d

ALGORITHME AIRE



PROGRAMME AIRE

```
#include <stdio.h>
void main()
{
    float aire, pi, rayon;
    rayon = 3.7;
    pi = 3.14159;
    aire = pi*rayon*rayon;
    printf("Aire est %f\n", aire);
}
```

LA FONCTION SCANF()

Définition :

La fonction scanf() :

- appartenant à la bibliothèque **stdio.h**
- permet de lire des données au clavier et de les affecter à des variables.
- scanf (scan : formatted) peut être utilisée pour la saisie des données formatées (lettre, chiffre ou chaîne de caractère)
C'est une fonction variadique (peut prendre un nombre variable de paramètres)
La fonction retourne le nombre de variable affectées par la saisie
=> Permet de vérifier si la saisie s'est bien passée

LA FONCTION SCANF()

Syntaxe :

```
scanf("format_variable1 format_variable 2 ....format_variable n", &variable_1,  
      &variable_2, ..., & variable_n);
```

LA FONCTION SCANF() EXEMPLE

```
char alpha;
```

```
int i;
```

```
float r;
```

```
scanf("%c",&alpha);  $\Longrightarrow$  /* saisie d'un caractère */
```

```
scanf("%d",&i);  $\Longrightarrow$  /* saisie d'un nombre entier */
```

```
scanf("%x",&i);  $\Longrightarrow$  /* saisie d'un nombre entier en hexadécimal*/
```

```
scanf("%f",&r);  $\Longrightarrow$  /* saisie d'un nombre réel */
```


LA FONCTION SCANF() EXEMPLE

```
#include <stdio.h>
#include <string.h>
void main()
{
    // declaration des variables
    int V1,V2,V3;
    char S1[30], S2[30];
    int Somme ;
    // lecture des données
    printf("votre nom : "); scanf("%s", S1);
    printf("votre prenom : ") ; scanf("%s", S2);

    // lecture et traitement
    printf("la saisie des donnees :\n ");
    Somme=scanf("%d%d%d",&V1,&V2,&V3);
    //affichage

    printf("%s %s vous avez saisi %d nombres ",S1, S2, Somme);

}
```

```
votre nom : ELFILALI
votre prenom : SANAA
la saisie des donnees :
3
4
5
ELFILALI SANAA vous avez saisi 3 nombres
```

AIRE D UN CERCLE (AMÉLIORATION)

```
/* Premier programme en C */  
#include <stdio.h>  
void main()  
{  
    float aire, pi, rayon;  
    // rayon = 3.7;  
    printf("Entrez une valeur pour rayon : ");  
    scanf("%f", &rayon);  
    pi = 3.14159;  
    aire = pi*rayon*rayon;  
    printf("Aire est %f\n", aire);  
}
```

AIRE D'UN CERCLE (AMÉLIORATION)

```
printf("Entrez une valeur pour rayon : ");
```

```
// Ce printf n'a pas de \n (nouvelle ligne)
```

```
scanf("%f", &rayon);
```

```
//scanf prend la valeur réelle entrée (%f) et l'assigne à la  
variable rayon.
```

AIRE D'UN CERCLE (AMÉLIORATION) EXÉCUTION DU PROGRAMME

Entrez une valeur pour rayon : 3.7

Aire est 43.008369

La fonction Scanf() Exemple

Saisie de plusieurs variables.

```
{
```

```
float a, b, c, det ;
```

```
printf("Donnez les valeurs de a,b et c : ");
```

```
/* Saisie de a, b, c */
```

```
scanf("%f %f %f",&a,&b,&c);
```

```
/* Calcul du déterminant */
```

```
det = b*b- 4*a*c;
```

```
printf("Le déterminant = %.2f", det);
```

```
}
```

La fonction Scanf() Exemple

Tampon de scanf

Les informations tapées au clavier sont d'abord mémorisées dans un emplacement mémoire appelé *tampon* (**buffer**).

elles sont mises à la disposition de **scanf** après l'activation de la touche <Entrée>

➤ Pour les nombres :

scanf avance jusqu'au premier caractère différent d'un séparateur (espace, tabulation, retour à la ligne)

puis **scanf** prend en compte tous les caractères jusqu'à la rencontre d'un séparateur ou d'un caractère invalide ou en fonction du largeur.

➤ Pour les caractères %c :

- **scanf** prend le caractère courant qu'il soit séparateur ou non séparateur.

La fonction Scanf() Exemple

Tampon de scanf : résoudre le problème de la saisie des caractères

➤ Pour les caractères %c : il faut laisser un espace avant le % : exemple
`scanf(" %c",&ch);`

Un espace



Convertir le caractère
majuscule en minuscule
Et
Minuscule en majuscule

Exercice :

écrire un programme qui permet de convertir un caractère
saisi au clavier en majuscule et minuscule

Convertir le caractère majuscule en minuscule Et Minuscule en majuscule

```
#include <stdio.h>
void main(void)
{
    char ch;

    printf("veuillez saisir un caractere : "); scanf(" %c",&ch);

    printf("caractere saisi est : %c\t \n
           en maj = %c \t en min = %c\n", ch,toupper(ch), tolower(ch));
}
```

```
veuillez saisir un caractere : S
caractere saisi est : S
    en maj = S      en min = s
```

Convertir le caractère majuscule en minuscule Et Minuscule en majuscule

```
#include <stdio.h>
#include <ctype.h>
#include <time.h>
void main(void)
{

    char ch;
    time_t t = time(NULL);
    printf(" \t exemple cours : Exercice conversion      %s\n", asctime(localtime(&t)));
    printf(" \t ===== \n\n");
    printf("*****\n");
    printf("Realise par : SMI S3 \n");
    printf("Encadre par Professeur: S.ELFILAL\n");
    printf("*****\n");
    printf("veuillez saisir un caractere : "); scanf(" %c",&ch);

    printf("caractere saisi : %c\t maj toupper= %c\n", ch,toupper(ch));
    printf("caractere saisi : %c\t min tolower= %c\n", ch, tolower(ch));
    putchar('\n');

}
```

La moyenne de deux nombres entrés au clavier

Exercice :

écrire un programme qui permet de calculer la moyenne de deux nombres réels entrés au clavier et l'afficher :

La moyenne de deux nombres entrés au clavier

Exercice : écrire un programme qui permet de calculer la moyenne de deux nombres réels entrés au clavier et l'afficher :

```
#include <stdio.h>
void main()    // Point d'entrée du programme
{
    /*Début du programme
    */
    float x, y;
    printf("\t\t\tCalcul de la moyenne\n\n");           /*Affiche le titre*/
    printf("Entrez le premier nombre : ");
    scanf("%f", &x);                                     /*Lecture du premier nombre*/
    printf("Entrez le deuxième nombre : ");
    scanf("%f", &y);                                       /*Lecture du deuxième nombre*/
    printf("La valeur moyenne de %.2f et de %.2f est %.2f\n",x, y,(x+y)/2);
} // Fin du programme
```

La Programmation en langage C

Chapitre 3: les instructions de contrôle

La cible : MIP S3

Pr S.ELFILALI

Introduction

Les instructions de contrôle

servent à

- contrôler le déroulement de l'enchaînement des instructions à l'intérieur d'un programme

peuvent être

- des instructions conditionnelles
- des instructions itératives.

Les instructions conditionnelles

Les instructions conditionnelles

Les instructions conditionnelles permettent de :

- réaliser des tests
- et suivant le résultat de ces tests
 - d'exécuter des parties de code différentes.

C offre deux types d'instructions conditionnelles :

- l'instruction **if**
- l'instruction **switch**

Instruction conditionnelle if

l'instruction
if-else

```
if ( <expression>) <instruction1>  
if ( <expression>) <instruction1>  
else <instruction2>
```


Instruction conditionnelle if

Description :

Instruction1 et
instruction2 :

- simple
- bloc
- instruction structurée

Si le résultat de
l'expression est « vrai »

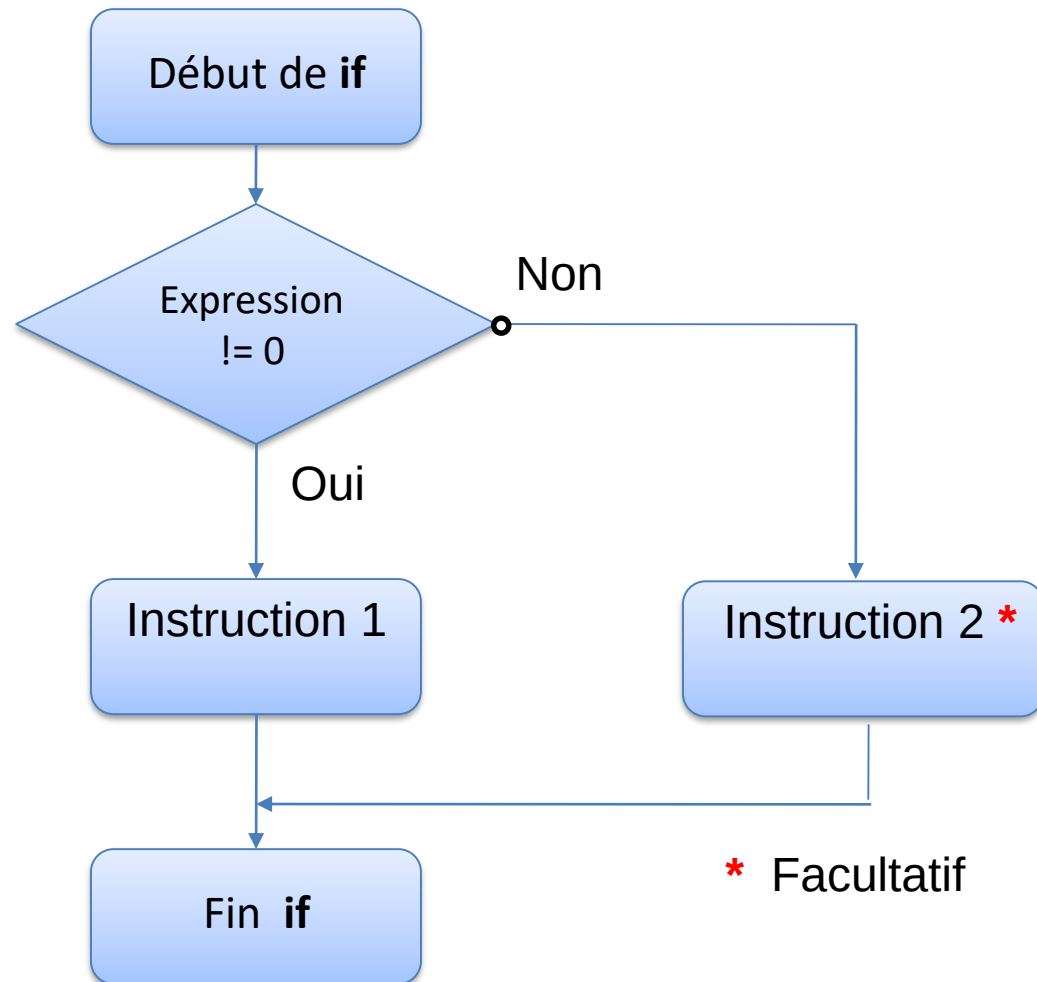
- on exécute l'instruction *<instruction1>*,

Dans le cas contraire

- on exécute l'instruction *<instruction2>* si elle existe.

Puis, le programme passe à l'exécution de l'instruction suivante.

Diagramme syntaxique du if :



À quoi sert l'instruction if ?

Exemple1 :

```
//declaration des variables
```

```
int qte_cmd ;
```

```
float prix, remise=0 ;
```

```
//lecture des donnees
```

```
printf("La quantité commandée : ") ; scanf("%d",&qte_cmd) ;
```

```
printf("Prix unitaire : ") ; scanf("%f",&prix) ;
```

```
//traitement
```

```
if(qte_cmd > 100) remise = 0.1;
```

```
//affichage des resultats
```

```
printf("Prix à payer : %.2f",prix*qte_cmd*(1 - remise) ) ;
```

Exemple2 :

```
//declaration des variables
```

```
char car ;
```

```
//lecture des donnees
```

```
printf("Tapez une lettre minuscule non accentuée : ");  
scanf(" %c",&car);
```

```
//traitement et affichage des resultats
```

```
if (car == 'a' || car == 'e' || car == 'i' || car == 'o' || car == 'u' || car == 'y')  
printf("la lettre %c est une voyelle",car);  
else  
printf("la lettre %c est une consonne",car);
```

Exemple3 :

```
//declaration des variables
```

```
.....
```

```
//lecture des donnees
```

```
.....
```

```
.....
```

```
//traitement et affichage des resultats
```

```
if (qte_cmd <= qte_stock)
{
    printf("Prix total = %.2f", qte_cmd*prix);
    qte_stock -= qte_cmd;
}
else
    printf("\aLa quantité commandée est sup. à la quantité en stock ! ");
```

Exercice 1 :

L'utilisateur saisit un caractère, le programme teste s'il s'agit d'une lettre majuscule, si oui il affiche cette lettre en minuscule, sinon il affiche un message d'erreur.

. (tolower, toupper)

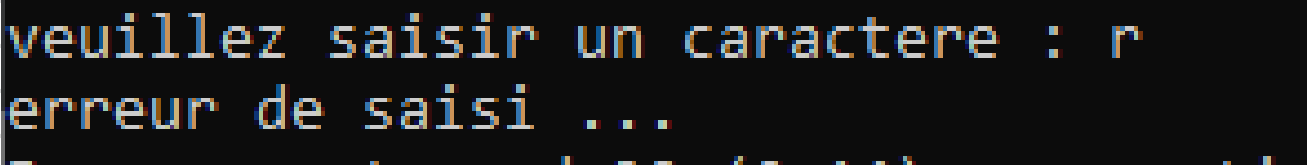
Solution :

```
#include <stdio.h>
void main(void)
{
    char ch;

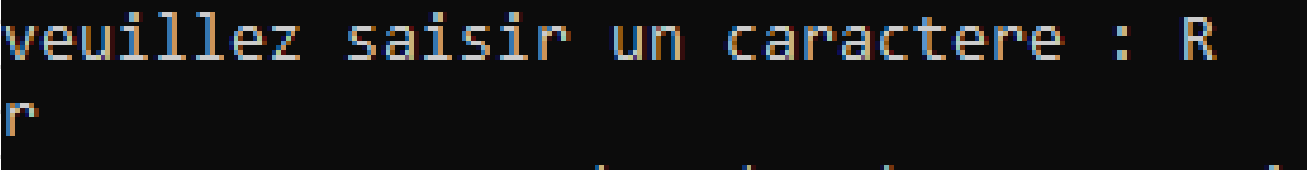
    printf("veuillez saisir un caractere : "); scanf("%c",&ch);

    if(ch==toupper(ch)) printf("%c",tolower(ch).);
    else printf("erreur de saisi ... ");

}
```



```
veuillez saisir un caractere : r
erreur de saisi ...
```



```
veuillez saisir un caractere : R
r
```

Exercice 2 :

Quel résultat affiche le programme suivant :

```
#include<stdio.h>
void main()
{
    int x = 3 ;

    if (x > 0)
    {
        x = -x ;
        printf("x = %d ", x);
    }
    if (x == 4) printf("x = %d", x);
}
```

Solution :

.....

.....**x=-3**

Quel résultat affiche le programme suivant :

```
#include<stdio.h>
void main()
{
    int x = 4 ;

    if (x > 0)
    {
        x = -x ;
        printf("x = %d ", x);
    }
    if (x == 4) printf("x = %d", x);
}
```

Solution :

.....**x=-4**

Imbrication d'instructions if

Il est possible d'imbriquer une instruction **if** à l'intérieur d'une autre.

Syntaxe :

```
if (<expression1>) if (<expression2>) <instruction1> else <instruction2>
```

Exemple :

```
if (a < b) if (c < b) z = b ; else z = a ;
```

Question : A quel « **if** » se rapporte « **else** » ?

Règle : le **else** se rapporte au dernier **if** rencontré auquel

un **else** n'a pas encore été attribué sauf si on utilise les accolades.

Exemple

La bonne mise en page du code ci-dessus est :

```
if (a < b)
  if (c < b)
    z = b ;
  else
    z = a ;
```

On peut mettre des { } pour rendre les choses plus claires.

```
if (a < b)
{
  if (c < b)
    z = b ;
  else
    z = a ;
}
```

Si l'on veut que « else » se rapporte au premier « if » :

```
if (a < b)
{
  if (c < b)
    z = b ;
}
else
  z = a ;
```

L'instruction if-else if

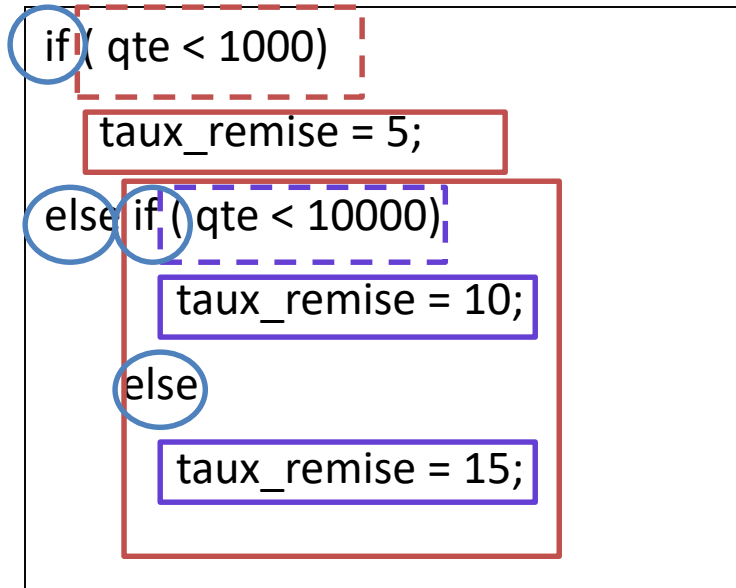
Il est fréquemment utile d'utiliser une série d'instructions **if-else if** pour répondre à une situation dans laquelle les choix sont multiples.

Syntaxe :

```
if (<expression1>) <instruction1>  
else if (<expression2>) <instruction2>  
else if (<expression3>) <instruction3>  
...  
[else <instruction_n+1>]
```

NB : Chaque instruction est associée à une condition et le dernier else, s'il existe, correspond au cas où aucune condition n'est vraie.

Exemples :



Exemples :

```
#include<stdio.h>
```

```
void main()
```

```
{
```

```
    char c;
```

```
    printf("Entrez une lettre non accentuée : ");
```

```
    scanf(" %c", &c);
```

```
    if (c >= 'A' && c <= 'Z')
```

```
        printf("Cette lettre en minuscule est : %c", c+32);
```

```
    else if (c >= 'a' && c <= 'z')
```

```
        printf("Cette lettre en majuscule est : %c", c-32);
```

```
    else
```

```
        printf("Erreur ! Ce n'est pas une lettre non accentuée");
```

```
}
```

Exercice 3 :

Ecrivez un programme qui permet de saisir une valeur de type entière et indiquez à l'utilisateur si celle-ci est positive, négative ou nulle.

Solution :

This image shows a full page of a document template designed for writing. It features a series of evenly spaced, horizontal grey lines that run across the entire width of the page. The lines are thin and light, providing a guide for text alignment without being distracting. There are no margins, headers, footers, or other markings present on the page.

L'opérateur ?:

Le langage C offre la possibilité de remplacer l'instruction conditionnelle **if-else** dans le cas d'une affectation conditionnelle de variable par l'opérateur conditionnel « ?: » et qui a l'avantage de pouvoir être intégré dans une expression.

Syntaxe :

expr1 ? expr2 : expr3

On évalue **expr1**:

Si **expr1** est vraie

- Résultat est la valeur de **<expr2>**

Si **expr1** n'est pas
vraie,

- Résultat est la valeur de **<expr3>**

Examples :

```
int x = 5, y = 4, max;  
if (x>y)  
    max=x;  
else  
    max=y;  
printf("Le max est : %d",max) ;
```



```
int x = 5, y = 4, max;  
max = (x>y) ? x : y;  
printf("Le max est : %d",max) ;
```



```
int x = 5, y = 4;  
printf("Le max est : %d",(x>y) ? x : y) ;
```

Instruction conditionnelle switch

À quoi sert l'instruction switch?

L'instruction conditionnelle **switch** permet de remplacer plusieurs if-else imbriqués lorsqu'il s'agit d'effectuer un choix multiple.

Exemples :

switch (expression)

```
{  
  case expression_const_1 :  
    instruction_1;  
    [break;  
  case expression_const_2 :  
    instruction_2;  
    [break;  
    ....  
  
  case expression_const_n :  
    instruction_n;  
    [break;  
  [default:  
    instruction_par_défaut;  
}
```

NB :

expression et
expression_const_i

- ni de type réel
- ni de type chaîne de caractère.

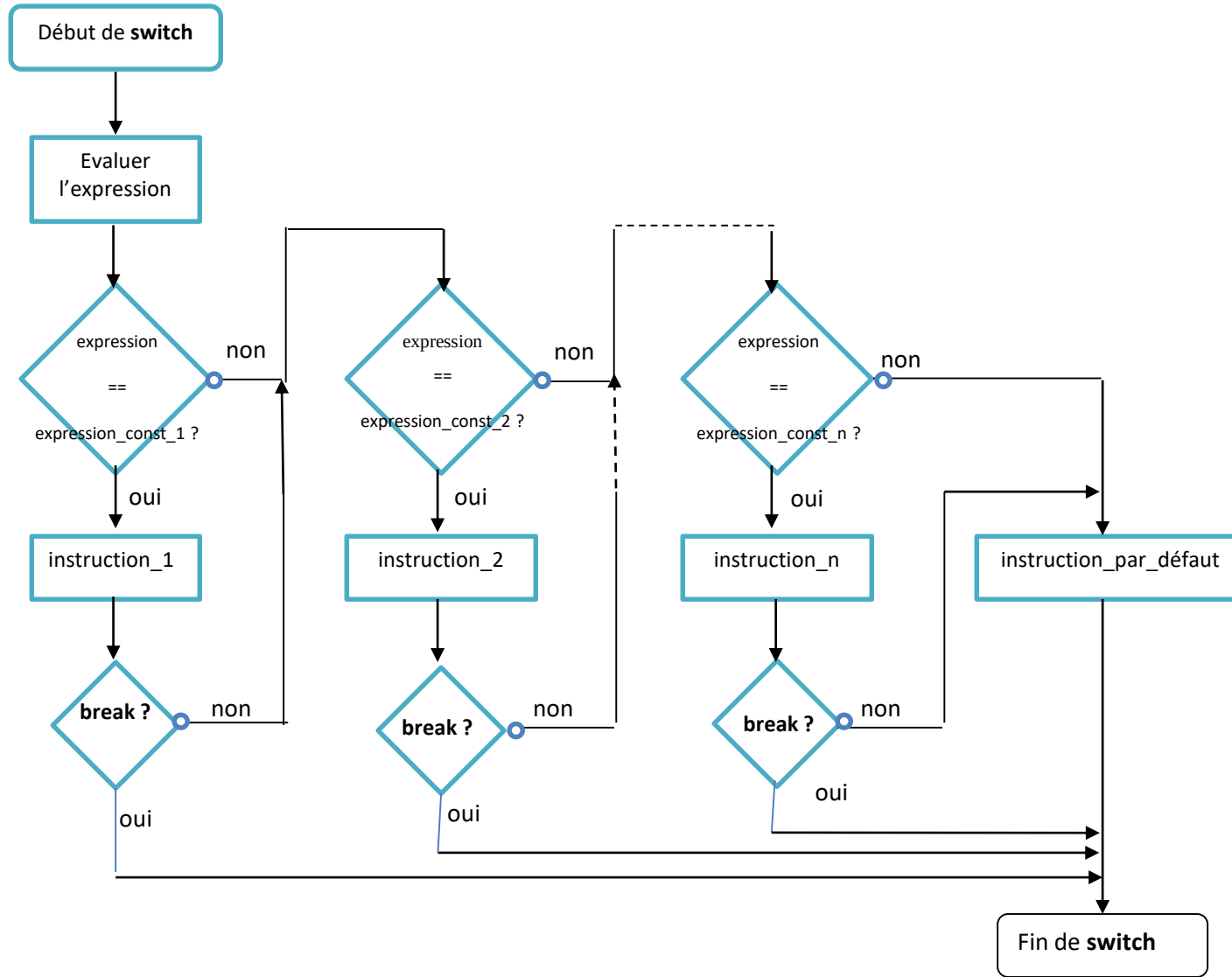
expression_const_i

- doivent obligatoirement être distinctes.

break et default

- sont optionnelles.

Diagramme syntaxique de switch :



Exemples :

```
void main()
{
    int jour;
    printf("Entrez le numéro d'un jour de la semaine (1 à 7) : "); scanf("%d",&jour);
    printf("Le jour %d de la semaine est le ",jour);
    switch (jour)
    {
        case 1 :      printf("DIMANCHE");
                       break;

        case 2 :      printf("LUNDI");
                       break;

        .....

        case 7 :      printf("SAMEDI");
                       break;

        default: printf("Erreur!");
    }
}
```

Exemples :

```
void main()
{
    int jour;
    printf("Entrez le numéro d'un jour de la semaine (1 à 7) : "); scanf("%d",&jour);
    printf("Le jour %d de la semaine est le ",jour);
        if (jour == 1)
            printf("DIMANCHE");
        else if (jour == 2)
            printf("LUNDI");
            else if (jour == 3)
                printf("MARDI");
                .....
            else if (jour == 7)
                printf("SAMEDI");
            else
                printf("Erreur!");
}
```

Instructions itératives

Les instructions itératives ou boucles sont réalisées à l'aide d'une des trois instructions de contrôle suivantes :

Instructions itératives

do-while

while

for

Instruction do-while <faire-tant que> :

La boucle *do-while* permet de répéter une instruction ou un bloc d'instructions, un certain nombre de fois, tant qu'une condition est vérifiée.

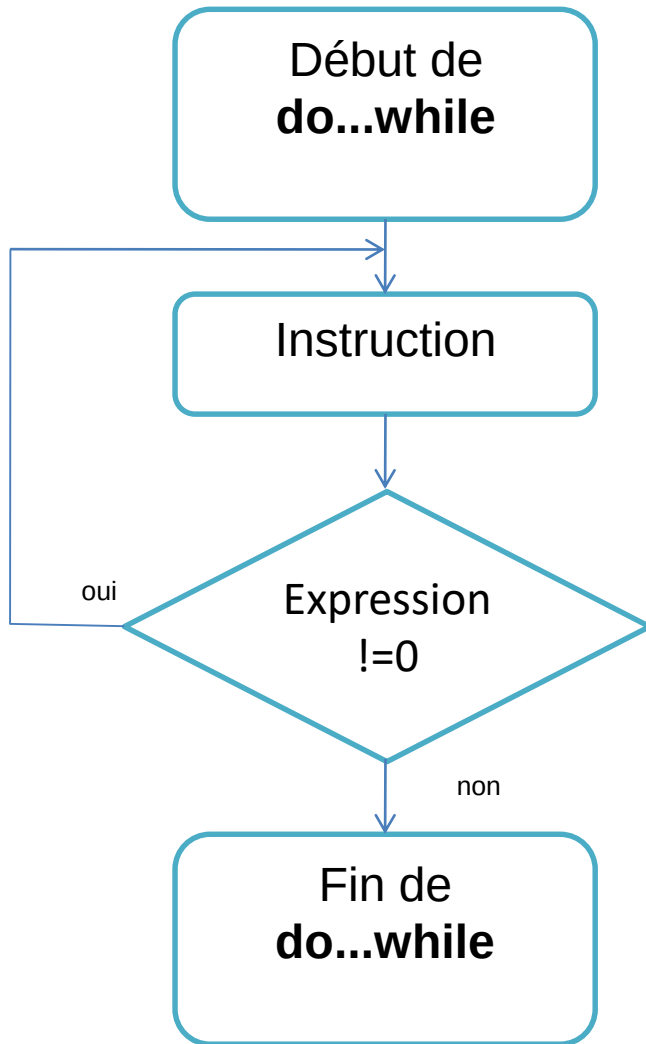
Syntaxe

do

<instruction> /*ou <bloc d'instructions> */

while (<expression>);

Diagramme syntaxique do-while :



Remarques :

L'<instruction> est exécutée au moins une fois,

- car le test de l'<expression> s'effectue à la fin de la boucle.

Il faut s'assurer que l'<instruction> modifie la valeur de l'<expression>

- si on veut éviter d'être pris dans une boucle sans fin.

Exemples :

```
void main ()
{
int compteur = 1;

do
{
    printf("La valeur du compteur vaut : %d\n", compteur);
        compteur++ ;
}while (compteur <=10);

printf("La valeur finale du compteur vaut : %d",compteur);
}
```

Exemples :

```
#define PI 22/7.0
void main ()
{
    float rayon ;
    do
    {
        printf("Donnez le rayon : ") ;scanf("%f",&rayon);
        if(rayon < 0) printf("\aLe rayon doit être positif\n") ;
    }while (rayon < 0);
    printf("Aire = %.2f",rayon*rayon*PI);
}
```


Exemples :

```
#include <stdio.h>
void main ()
{
    char rep ;
    do
    {
        ....
        printf("Vous-voulez continuer (o/n) : " ) ;
        scanf(" %c",&rep); /* rep = getche(); */

    }while (rep == 'o' || rep == 'O');
}
```

L'instruction while <tant que>

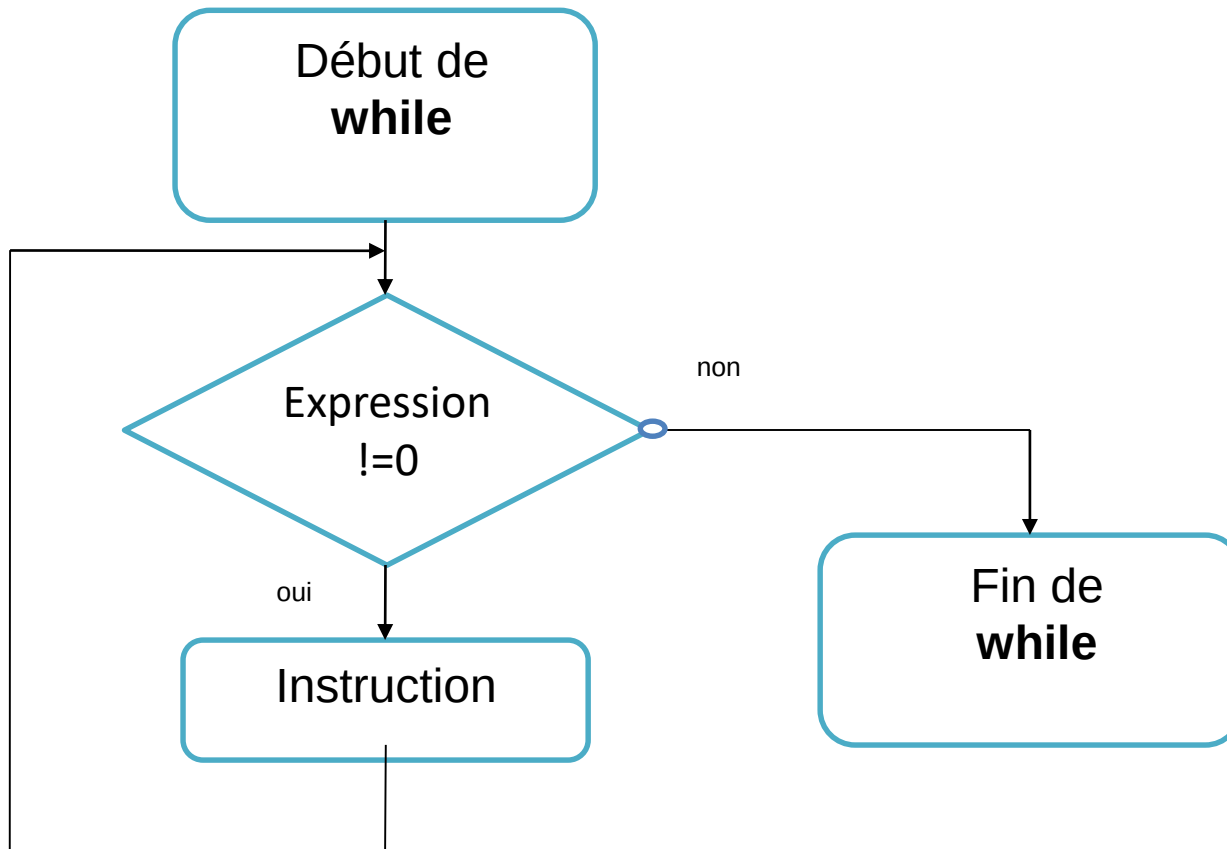
L'instruction **while** se comporte comme la boucle **do-while**, à la différence près que l'<expression> est évaluée en début de boucle. Il y a donc possibilité que l'<instruction> à répéter ne soit jamais exécutée, si le premier test n'est pas vérifié.

Syntaxe

While (<expression>)

<instruction> ; /*ou <bloc d'instructions> */

Diagramme syntaxique de while :



Examples :

```
#include <conio.h>
void main ()
{
    int color = 1;
    while (color <= 15)
    {
        textcolor(color);
        cprintf("Couleur N  %d\n\r", color++);
    }
}
```



```
HANDLE hStdOut = GetStdHandle(STD_OUTPUT_HANDLE);  
CONSOLE_SCREEN_BUFFER_INFO csbi;
```

```
    //We use csbi for the wAttributes word.
```

```
if(GetConsoleScreenBufferInfo(hStdOut, &csbi))  
{  
    //Mask out all but the background attribute, and add in the foreground  
color  
    wColor = (csbi.wAttributes & 0xF0) + (ForgC & 0x0F);  
    SetConsoleTextAttribute(hStdOut, wColor);  
}  
}
```

```
void main ()  
{  
    int color =1;  
    while(color<=15)  
    {  
        SetColor(color);  
        printf("couleur N  %d \n\r", color++);  
    }  
}
```

Exercices :

Exercice 4 :

Ecrire un programme qui demande un entier positif n et qui affiche la somme des n premiers entiers.

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int s=0, n,i;
    printf("donner un entier
positif : "); scanf("%d",&n);
    i=0;
    do
    {
        s+=i;
        i++;
    } while (i <= n);

    printf("la somme est :%d
",s);

    return 0;
}
```

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int s=0, n,i;
    printf("donner un entier
positif : ");
    scanf("%d",&n);
    i=0;

    while (i <= n)
    {
        s+=i;
        i++;
    }
    printf("la somme est :%d
",s);

    return 0;
}
```

```
#include <stdlib.h>
#include <stdlib.h>

int main()
{
    int s=0, n,i;
    printf("donner un entier
positif : ");
    scanf("%d",&n);

    for(i=0;i<=n;i++)
        s+=i;

    printf("la somme est :%d
",s);

    return 0;
}
```

Exercices :

Exercice 5

Ecrire un programme qui calcule la somme des nombres positifs donnés par l'utilisateur, le programme lira des nombres tant qu'ils seront positifs.

Solution :

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int s=0, n;
    do
    {
        printf("donner un entier positif : "); scanf("%d",&n);

        if(n>=0)    s+=n;

    } while (n >= 0);
    printf("la somme est :%d ",s);

    return 0;
}
```

Instruction itérative for <Pour> :

Syntaxe

```
for ([expression_1];[expression_2];[expression_3])  
<instruction>; /*ou <bloc d instructions>*/
```

Description :

expression_1 : est évaluée une seule fois. Elle sert à initialiser les données de la boucle.

expression_2 : est la condition d'exécution de l'<instruction> à répéter. Elle est évaluée et testée avant chaque parcours de la boucle. Si son résultat est VRAI alors l'<instruction> est exécutée sinon la boucle est terminée

expression_3 : est évaluée après l'exécution de l'<instruction> à répéter. Elle est utilisée pour mettre à jour les données de la boucle.

Instruction itérative for <Pour> :

```
for ([expression_1];[expression_2];[expression_3])  
    <instruction>; /*ou <bloc d'instructions>*/
```

Description :

➤ La boucle **for** s'utilise avec **trois** expressions, **séparées par des points virgules** :

expression_1 :

- est évaluée une seule fois, au début de l'exécution de la boucle .
- Elle sert à initialiser les données de la boucle.

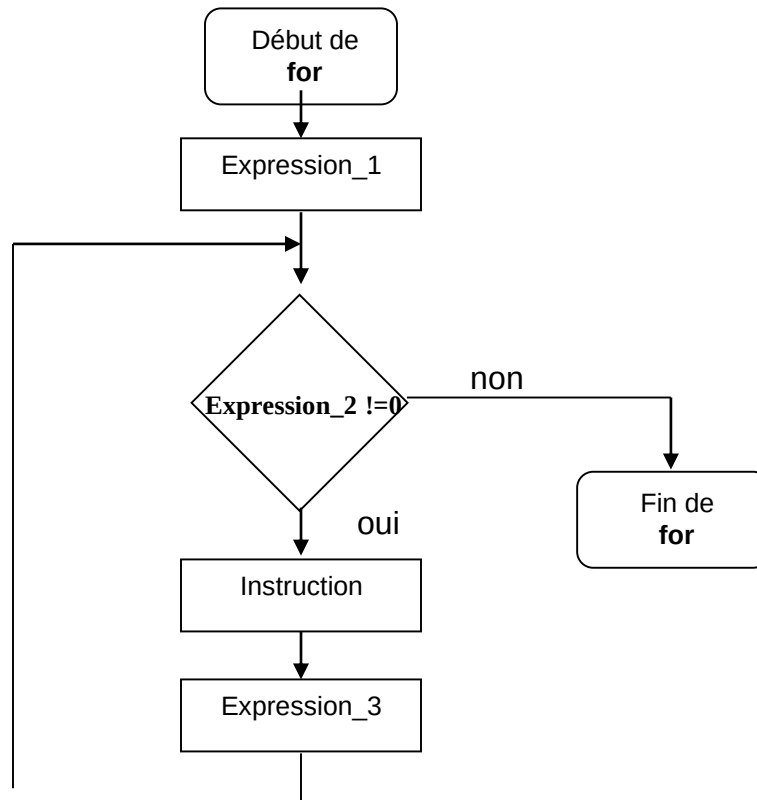
expression_2 :

- est la condition d'exécution de l'<instruction> à répéter.
- Elle est évaluée et testée avant chaque parcours de la boucle.
- Si son résultat est VRAI alors l'<instruction> est exécutée
- sinon la boucle est terminée.

expression_3 :

- est évaluée après l'exécution de l'<instruction> à répéter.
- Elle est utilisée pour mettre à jour les données de la boucle.

Diagramme syntaxique



Instruction itérative for <Pour> :

Remarques :

- La boucle for est équivalente à la structure suivante :

```
expression_1;  
    while (expression_2)  
    {  
        instruction ;  
        expression_3;  
    }
```

- Par rapport à la boucle while, la boucle for met en valeur l <expression_1> et l <expression_3>.
- Expression_1, expression_2 et expression_3 sont facultatives.
- Si l <expression_2> est omise alors la boucle est infinie.
- Quand tout le traitement peut être spécifié dans les trois expressions du for, l instruction à répéter peut se réduire à une instruction vide.

Exemples :

```
#include <stdio.h>
void main ()
{
    int m,i ;
    printf("Donnez le multiplicande compris entre 1 et 9 : ") ;
    scanf("%d",&m);
    printf("\n\t\t\tTable de multiplication par %d\n",m) ;
    for (i=1; i <= 10 ; i++)
        printf("%2d x %2d = %2d\n",m,i,m*i) ;
}
```

Exercices :

Exercice 6 :

Ecrire un programme qui affiche la somme des **n** premiers termes d'une progression arithmétique de raison **r** et de premier terme **m**.

Par exemple, si **n** = 7, **r** = 3 et **m** = 23 alors

$$\text{somme} = 23 + 26 + 29 + 32 + 35 + 38 + 41 = \mathbf{224}$$

$$= (m+0*r)+(m+1*r)+(m+2*r) + (m+3*r) + \dots (m+n*r)$$

Solution :

```
#include <stdio.h>
#include <stdlib.h>
#include <conio.h>
#include <windows.h>
void main()
{
    int i,n,m,r,s;

    printf("donner le premier terme de la suite : ");scanf("%d",&m);
    printf("donner la raison de la suite : ");scanf("%d",&r);
    printf("donner le nombre de n elements de la suite : ");scanf("%d",&n);

    s=0;
    for (i=0;i<n;i++)

        s=s+(m+i*r);

    printf("la somme de %d premier terme de la suite de la raison %d et du premier terme  %d est : %d ", n,r,m,s);
    getch();
}
```

Remarque sur les instructions itératives

Les différentes boucles peuvent être imbriquées.

```
#include <stdio.h>
#include <conio.h>
void main ()
{
    int m,n ;

    for (m=1; m <= 9 ; m++)
    {
        clrscr();
        printf("\t\t\tTable de multiplication par %d\n\n",m) ;
        for (n=1; n <= 10 ; n++)
            printf("\t\t\t\t\t%2d x %2d = %2d\n",m,n,m*n);
        getch();
    }
}
```

Lorsque l'on imbrique des instructions for, il faut veiller à ne pas utiliser le même compteur pour chacune des instructions.

Les instructions de rupture de séquences

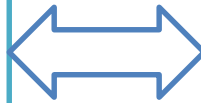
Ces instructions permettent de rompre le déroulement séquentiel d'une suite d'instructions.

L'instruction break

L'instruction **break** provoque le passage à l'instruction qui suit immédiatement le corps de la boucle *while*, *do-while*, *for* ou *switch*.

```
#include <stdio.h>

void main()
{
    int i;
    for(i=0; ;i++)
    {
        printf("La valeur du compteur vaut :
%d\n", i);
        if(i == 10) break;
    }
}
```



```
#include <stdio.h>

void main()
{
    int i = -1;
    do
    {
        printf("La valeur du compteur vaut :
%d\n", ++i);
        if(i == 10) break;
    } while(1) ;
}
```

L'instruction break

Que se passe -t-il avec le code suivant?

```
#include <stdio.h>
void main()
{
    int i;
    for(i=0;i<=20 ;i++)
    {
        printf("La valeur du compteur vaut :
%d\n", i);
        if(i == 10) break;
    }
}
```


L'instruction break

Remarques :

- L'instruction break ne peut être utilisée que dans le corps d'une boucle ou d'un switch.
- L'action de break ne s'applique qu'à la boucle la plus intérieure dans laquelle elle se trouve.
- Elle ne permet pas de sortir de plusieurs boucles imbriquées.

L'instruction goto

Cette instruction permet de se brancher (inconditionnellement) à une étiquette (identificateur) à l'intérieur de la même fonction.

Sa syntaxe est la suivante :

```
goto étiquette ;
```

Et la déclaration d'une étiquette se fait de la manière suivante

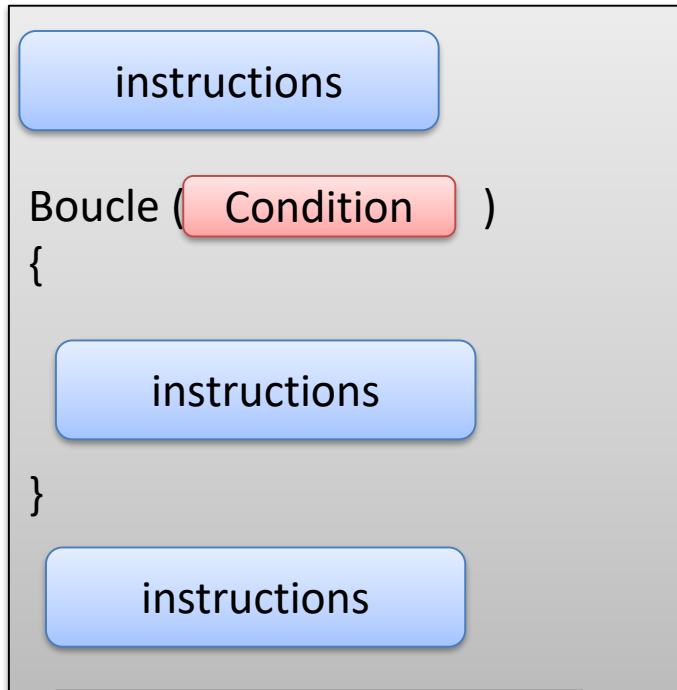
```
étiquette :  
    instruction ;
```

Examples :

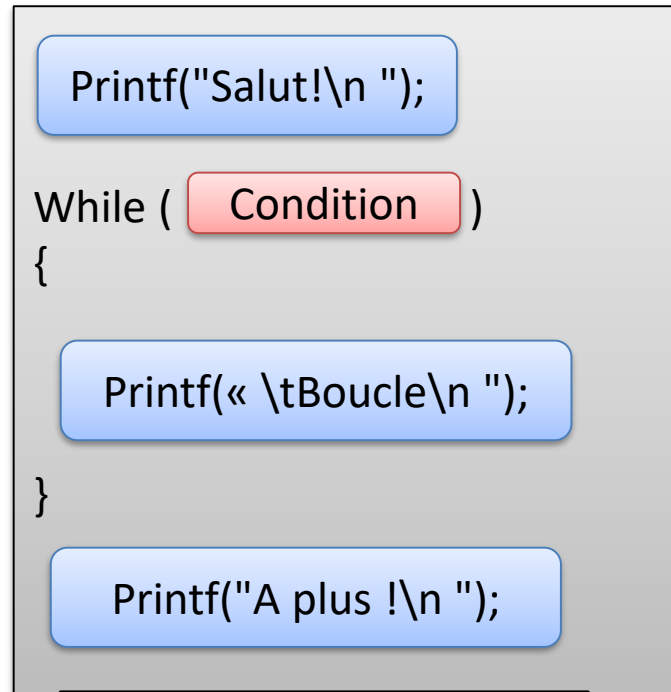
```
void main()
{
    int i,j,k;
    for(i=1 ;i<=5 ;i++)
        for(j=1;j<=5;j++)
            for(k=1;k<=5;k++)
            {
                printf("i = %d\t j = %d\t k = %d \n",i,j,k);

                if (i*j*k== 40) goto sortie;
            }
sortie : printf("Fin du programme") ;
}
```

Résumons : Les boucles



Structure



Exemple

Salut !

Boucle
Boucle
Boucle
Boucle
Boucle

A plus !

Boucles en C : while, do while et for

Résumons : Les boucles

C'est quoi une boucle ?

Boucle c'est un ensemble d'instructions qui est exécuté un certain nombre de fois tant que la condition liée est vraie

Résumons : Les boucles

La boucle while

```
While ( Condition )  
{  
    instructions  
}
```

Structure

```
Int var =0;  
While ( var < 5 )  
{  
    Printf("%d\n" , var);  
    Var++;  
}
```

Exemple

0
1
2
3
4

Résumons : Les boucles

La boucle while

Exemple : compte à rebours

Passer sur codeblocks et faire un programme sans boucle et avec la boucle

Utiliser les fonctions sleep et générer le bip par `printf("\a");`

la fonction `Sleep("temps en millisecondes");`, permet de faire attendre le programme pendant un certains nombre de millisecondes avant de continuer à exécuter les instructions.

La boucle while

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int compte=10;
    printf("\a"); // bip
    getch();
    printf("explosion dans 10 s");

    printf("\ncompte à rebours ..... !\n");

    printf("\n%d",compte);
    compte--;
    sleep(1); // attente en millisecondes

    printf("\n%d",compte);
    compte--;
    sleep(1);

    printf("\n%d",compte);
    compte--;
    sleep(1);

    printf("\n%d",compte);
    compte--;
    sleep(1);

    printf("\nBOOOOOOMMM");
    return 0;
}
```


La boucle while

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int compte=10;
    printf("\a");
    getch();
    printf("explosion dans 10 s");

    printf("\ncompte à rebours ..... !\n");

    while (compte>0)
    {
        printf("\n%d",compte);
        compte--;
        sleep(1);
    }

    printf("\a");

    printf("\nBOOOOOO MMM");
    return 0;
}
```

Résumons : Les boucles

La boucle do while

```
do  
{  
    instructions  
} While ( Condition );
```

Structure

```
Int var =6;  
do  
{  
    Printf("%d\n" , var);  
    Var++;  
} While ( var < 5 );
```

Exemple

6

Résumons : Les boucles

La boucle do while

Exemple : contrôle de saisie

Passer sur codeblocks et faire un programme qui demande à l'utilisateur de saisir un nombre entre 1 et 10

Résumons : Les boucles

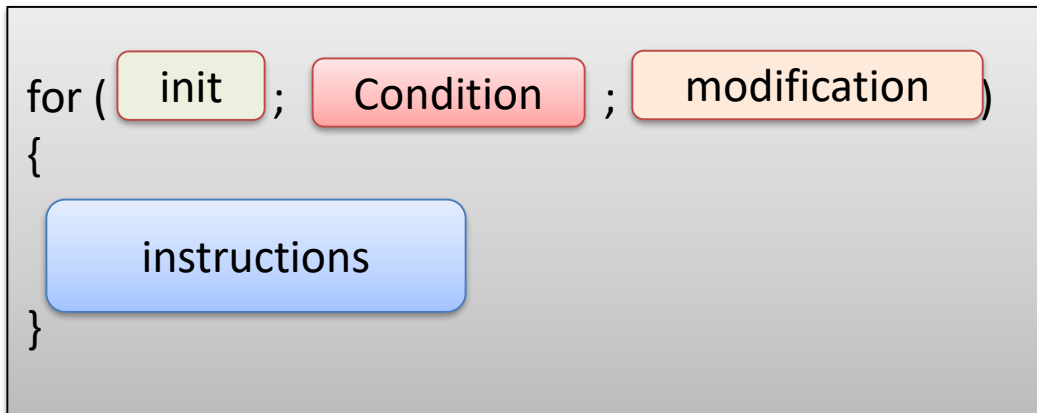
La boucle do while

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int saisie_utilisateur = 0;
    do
    {
        printf("donner un nombre entre 1 et 10 : ");
        scanf("%d",&saisie_utilisateur);
    }
    while (saisie_utilisateur <1 || saisie_utilisateur>10);
    return 0;
}
```

Résumons : Les boucles

La boucle for



Structure

```
for (int var =0 ; var<5 ; var++ )  
{  
    printf("%d\n",var);  
}
```

Exemple

0
1
2
3
4

```
int var =0 ;  
while(var<5)  
{  
    printf("%d\n",var);  
    var ++ ;  
}
```

Equivalent

0
1
2
3
4

Résumons : Les boucles

La boucle for

Exemple : compte à rebours

Passer sur codeblocks et faire un programme avec la boucle for

Utiliser les fonctions sleep et générer le bip par `printf("\a");`

la fonction `Sleep("temps en millisecondes");`, permet de faire attendre le programme pendant un certains nombre de millisecondes avant de continuer à exécuter les instructions.

```
#include <stdio.h>
#include <stdlib.h>
```

```
int main()
{
int compteur = 10;
    printf("Explosion dans  ");
    for( compteur = 10 ; compteur>0 ; compteur--)
    {
        printf("\n%d", compteur);
    }
    printf("\n\na BOOM");
    return 0;

}
```

Règles d'or des boucles

Comment choisir la boucle qu'on va utiliser ?

Si on connaît le nombre d'itération du bloc d'instruction alors on utilise la boucle for

Si on connaît pas le nombre d'itération du bloc alors nous avons deux choix :

Si on veut exécuter au moins une fois le bloc d'instruction alors c'est la boucle « do .. While »

Et dans tous les autres cas , on utilise la boucle while

Règles d'or des boucles

Boucle infinie

C'est une boucle qui va s'exécuter à l'infini!

Cad que sa condition ne va jamais passer à faux!!
Le programme va se bloquer !!

Le seul moyen pour le débloquent : **Ctrl+C** ou **Alt F4**

Exemple

Apportez les modifications suivantes au programme et commentez :

1. Compteur ++
2. Modifier le type en short et compteur ++
3. Modifier la condition : >-99999 et compteur --

```
#include <stdio.h>
#include <stdlib.h>
int main()
{
    int compteur = 10;
    printf("Explosion dans ");
    for( compteur = 10 ; compteur>0 ; compteur --)
    {
        printf("\n%d", compteur);
    }
    printf("\n\na BOOM");
    return 0;
}
```

Règles d'or des boucles

Boucle infinie

Apportez les modification suivantes au programme et commentez :

1. **Compteur ++** : boucle infinie, on n'arrive pas à voir le max de ce type , on doit l'arrêter par Ctrl+C ... la condition peut être fautive au max de ce type
2. **Modifier le type en short et compteur ++** : il s'arrête au max du short qui est 32767 et puis il passe à la première valeur de type int avec une incrémentation et dans ce cas la condition n'est pas vérifiée et la boucle s'arrête !
3. **Modifier la condition : >-99999 et compteur --** : la condition est toujours vérifiée et il ne va jamais s'arrêter !!!!!

```
#include <stdio.h>
#include <stdlib.h>
int main()
{
    int compteur = 10;
    printf("Explosion dans ");
    for( compteur = 10 ; compteur>0 ; compteur --)
    {
        printf("\n%d", compteur);
    }
    printf("\n\n BOOM");
    return 0;
}
```

Exemple

Les boucles

Exercice : code PIN

- 1- Créer un programme qui demande à l'utilisateur de rentrer le code PIN à 4 chiffres. Si il trouve le bon code, afficher le message : téléphone déverrouillé sinon afficher un message d'erreur et redemander d'entrer le code.
- 2- Gérer un nombre maximum de tentatives: Exemple 3 tentatives

```
Code PIN: 1234
  Erreur, il vous reste 2 tentatives.
Code PIN: 0000
  Telephone deverrouille
```

- 1- On ne connaît pas le nombre d'itération mais on souhaite entrer dans la boucle au moins une fois. On utilise alors une boucle "Do While"
- 2- Créer une constante pour le nombre maximum de tentatives.

Les boucles

Solution : code PIN

```
#include <stdio.h>
#include <stdlib.h>
int main()
{
    const int CODE_PIN = 0000;
    const int MAX_TENTATIVE = 3;
    int saisie;
    int nbre_Tentative= MAX_
TENTATIVE;
    // lecture de la saisie

    do {
        printf("Code Pin : ");
        scanf("%d",&saisie);

        if(saisie==CODE_PIN)
        {
            printf("\tTelephone deverrouille\n");
        }
        else
        {
            nbre_Tentative --;
            printf("\tErreur , il vous reste %d tentatives \n", nbre_Tentative);
        }
    }
    while(saisie!=CODE_PIN && nbre_Tentative > 0 );
    return 0;
}
```

Code PIN: 1234
Erreur, il vous reste 2 tentatives.
Code PIN: 0000
Telephone deverrouille

Les boucles

Break et Continue

break : sortir de la boucle courante

<pre>int main() { int var ; printf("debut \n"); for (var =0 ; var<5 ; var++) { if(var ==2) { break; } printf("%d\n",var); } printf("fin \n"); return 0; }</pre>	<pre>debut 0 1 fin</pre>
--	--------------------------

break

continue : sauter a la fin de la boucle

<pre>int main() { int var ; printf("debut \n"); for (var =0 ; var<5 ; var++) { if(var ==2) { continue; } printf("%d\n",var); } printf("fin \n"); return 0; }</pre>	<pre>debut 0 1 3 4 fin</pre>
---	------------------------------

continue

Les boucles

Break et Continue : exemple

```
int main()
{
    const int CODE_PIN = 0000;
    const int MAX_TENTATIVE = 3;
    int saisie_utilisateur;
    int nbre_Tentative= 0;
    int compteur;
    for( nbre_Tentative= 1 ;nbre_Tentative<=MAX_TENTATIVE ; nbre_Tentative++)
    {
        printf("Code Pin : ");
        scanf("%d",&saisie_utilisateur);
        if(saisie_utilisateur<0)
        {
            printf("code utilisateur doit etre superieur a 0 \n");
            continue;
        }
        if(saisie_utilisateur==CODE_PIN)
        {
            printf("\t Bienvenue\n");
            break;
        }
        else
            printf("\t Code incorrect  \n");
    }
    return 0;
}
```

Les boucles

Les boucles imbriquées

Exercice : afficher toutes les tables de multiplication de 1 à 10

Aide : commencer par afficher une seule table de multiplication

Table de 1 1 x 1 = 1 1 x 2 = 2 1 x 3 = 3 1 x 4 = 4 1 x 5 = 5 1 x 6 = 6 1 x 7 = 7 1 x 8 = 8 1 x 9 = 9 1 x 10 = 10	Table de 2 2 x 1 = 2 2 x 2 = 4 2 x 3 = 6 2 x 4 = 8 2 x 5 = 10 2 x 6 = 12 2 x 7 = 14 2 x 8 = 16 2 x 9 = 18 2 x 10 = 20	Table de 3 3 x 1 = 3 3 x 2 = 6 3 x 3 = 9 3 x 4 = 12 3 x 5 = 15 3 x 6 = 18 3 x 7 = 21 3 x 8 = 24 3 x 9 = 27 3 x 10 = 30	Table de 4 4 x 1 = 4 4 x 2 = 8 4 x 3 = 12 4 x 4 = 16 4 x 5 = 20 4 x 6 = 24 4 x 7 = 28 4 x 8 = 32 4 x 9 = 36 4 x 10 = 40	Table de 5 5 x 1 = 5 5 x 2 = 10 5 x 3 = 15 5 x 4 = 20 5 x 5 = 25 5 x 6 = 30 5 x 7 = 35 5 x 8 = 40 5 x 9 = 45 5 x 10 = 50
Table de 6 6 x 1 = 6 6 x 2 = 12 6 x 3 = 18 6 x 4 = 24 6 x 5 = 30 6 x 6 = 36 6 x 7 = 42 6 x 8 = 48 6 x 9 = 54 6 x 10 = 60	Table de 7 7 x 1 = 7 7 x 2 = 14 7 x 3 = 21 7 x 4 = 28 7 x 5 = 35 7 x 6 = 42 7 x 7 = 49 7 x 8 = 56 7 x 9 = 63 7 x 10 = 70	Table de 8 8 x 1 = 8 8 x 2 = 16 8 x 3 = 24 8 x 4 = 32 8 x 5 = 40 8 x 6 = 48 8 x 7 = 56 8 x 8 = 64 8 x 9 = 72 8 x 10 = 80	Table de 9 9 x 1 = 9 9 x 2 = 18 9 x 3 = 27 9 x 4 = 36 9 x 5 = 45 9 x 6 = 54 9 x 7 = 63 9 x 8 = 72 9 x 9 = 81 9 x 10 = 90	Table de 10 10 x 1 = 10 10 x 2 = 20 10 x 3 = 30 10 x 4 = 40 10 x 5 = 50 10 x 6 = 60 10 x 7 = 70 10 x 8 = 80 10 x 9 = 90 10 x 10 = 100

Les boucles

Les boucles imbriquées

Exercice : afficher toutes les tables de multiplication de 1 à 10

Aide : commencer par afficher une seule table de multiplication

```
int main()
{
    int ligne = 0;
    int id_table = 1;

    for(id_table=1; id_table<=10; id_table++)
    {

        printf("table multiplication %d\n", id_table);
        printf("*****\n");
        for(ligne=1; ligne<=10; ligne++)
        {
            printf("\t%d x %d = %d\n", ligne, id_table, ligne*id_table);
        }
        printf("\n") ;
    }
    return 0;
}
```


Les boucles

Jeux kahoot

<https://create.kahoot.it/details/boucles/db371927-a176-4c1f-8811-2b3b73759cdb>

[Kahoot!](#)

Les boucles

TP : un jeu

Nous allons créer un petit jeu de type Le juste prix . Pour les plus jeunes, cela correspond à un ancien jeu télévisé qui consiste à demander, aux participants, de trouver le prix d'un objet. Ils font alors des propositions et le présentateur leur dit simplement "c'est plus" ou "c'est moins", jusqu'à ce que l'un d'entre eux trouve le bon prix. Mais bien sûr, il y a une limite de temps! Nous allons donc demander à l'utilisateur de trouver un nombre entier compris entre 1 et 100.

Il va alors pouvoir faire des propositions et pour chacune d'entre elles, nous allons afficher c'est plus ! ou c'est moins ! . lorsque l'utilisateur a trouvé le bon nombre alors on affiche un message de félicitation et on termine le programme. Le joueur a le droit à 10 tentatives.

```
...  
> 23  
c'est plus !  
> 30  
c'est moins !  
> 25  
Bravo! le nombre mystère est bien 25
```

Les boucles

TP : un jeu

Instructions

1- Créer la boucle de jeu

- 1.1 Demander à l'utilisateur de rentrer un entier pour trouver le nombre mystère.
- 1.2 Comparer la valeur saisie avec le nombre à trouver.
- 1.3 Gérer la fin de partie (le joueur a trouvé le bon nombre).

2- Ajouter un compteur qui indique combien de tentative a fait le joueur.

```
...  
tentative 3 > 23  
c'est plus !  
tentative 4 > 30  
c'est moins !  
tentative 5 > 25  
Bravo! le nombre mystère est bien 25
```

Les boucles

TP : un jeu

Pour aller plus loin

A- Modifier le code pour rendre le jeu plus difficile. pour cela, ajouter un nombre maximum de tentative.

B- Ajouter un menu à l'utilisateur en fin de partie lui permettant de faire une nouvelle partie ou quitter.

```
...  
tentative 3/10 > 23  
c'est plus !  
tentative 4/10 > 30  
c'est moin !  
tentative 5/10 > 25  
Bravo! le nombre mystère est bien 25  
  
Voulez-vous faire une nouvelle partie (1-oui, 2-non)  
> 2  
A bientôt !
```

Les boucles

TP : un jeu

De l'Aide

1. Faire une boucle avec pour condition le fait que la saisie utilisateur corresponde ou non au nombre à trouver.
2. Il faut créer une nouvelle variable initialisé a 0 et l'incrémenter à chaque tentative de l'utilisateur.

A- Créer une nouvelle constante qui comporte le nombre maximum de tentative. quand ce nombre est atteint, utiliser une des instruction suivante pour terminer la partie (break, continue).

B- Mettre la boucle de jeu dans une nouvelle boucle. la condition de cette nouvelle boucle doit vérifier le choix de l'utilisateur.

La Programmation en langage C

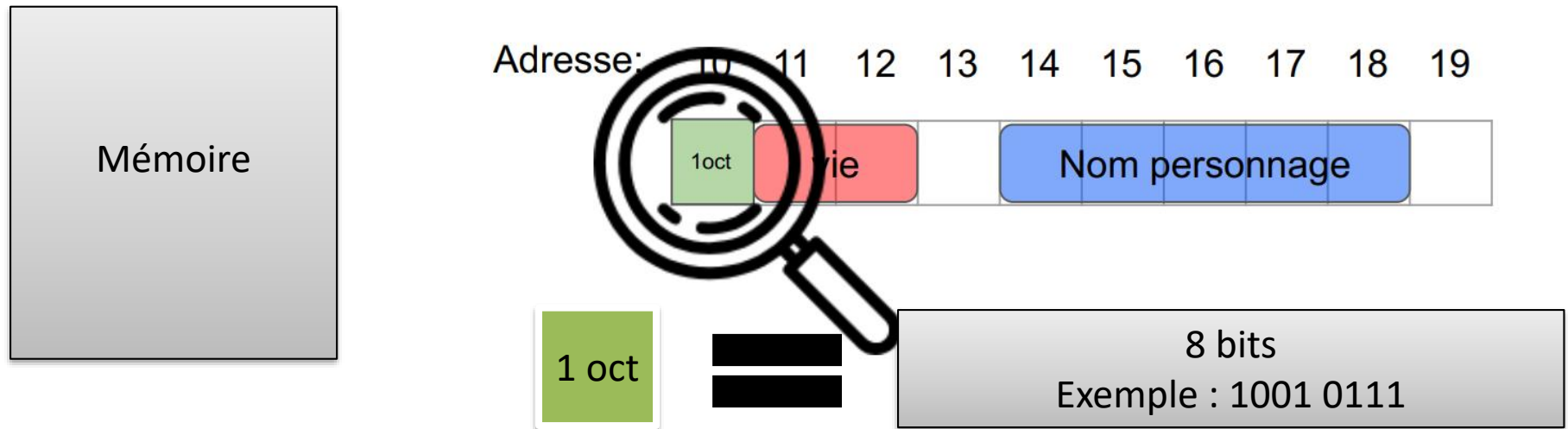
Chapitre 4 : les Pointeurs

La cible : MIP S3

Pr S.ELFILALI
sanaa.elfilali@univh2c.ma

Rappel :

Gestion de la mémoire et variable



- Le programme est exécuté depuis la RAM
- La mémoire est composée de cases continues et chacune de ces cases est représentée par un octet
- Un octet c'est 8 bits
- Ces cases sont identifiées de manières uniques à l'aide de leurs adresses
- Une variable est une boîte qui occupe un nombre entier de ces cases et qui permet de stocker les données en binaire
- La taille et l'interprétation dépend de leur type de la variable
- On peut accéder au contenu de la variable à l'aide de son nom qui sert comme label pour accéder à la variable

Notion de pointeur

- Un *pointeur* est une variable contenant l'adresse d'une autre variable d'un type donné.
- Il permet donc d'accéder **indirectement** à une variable.



Adresse: 10 11 12 13 14 15 16 17 18 19



Adresses et variables

Adresse: 10 11 12 13 14 15 16 17 18 19



```
int age_utilisateur = 25;
```

```
printf("Age utilisateur: %d\n", age_utilisateur);  
printf("Adresse: %d\n", &age_utilisateur);  
printf("Adresse: %p\n", &age_utilisateur);
```

```
Age utilisateur: 25  
Adresse: 10  
Adresse: 0x0A
```

Adresses et variables

Exemple sur codeblocks :

Afficher le contenu et l'adresse d'une variable en décimale et en hexadécimale

```
#include <stdio.h>
#include <stdlib.h>

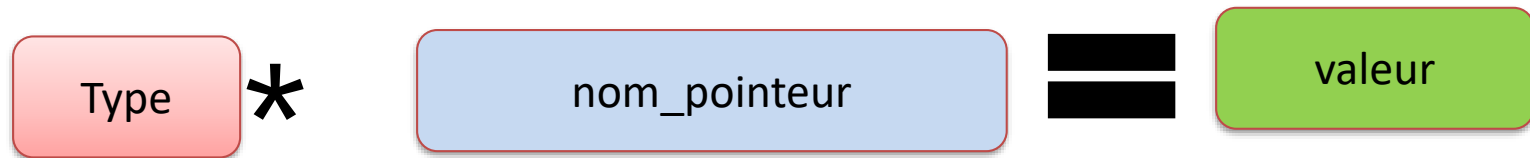
int main()
{
    int mon_int=10;
    printf("\nmon int = %d",mon_int );
    printf("\nAdresse de mon_int = %d",&mon_int);
    printf("\nAdresse de mon_int = %p",&mon_int);
    return 0;
}
```

```
mon int = 10
Adresse de mon_int = 1703736
Adresse de mon_int = 0019FF38
```

Représentation hexadécimale

Représentation	Alphabet	Exemple
Décimale (10)	0, 1, 2, 3, 4, 5, 6, 7, 8, 9	12
Binaire (2)	0, 1	1100
Hexadécimale (16)	0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F	C

Déclaration d'un pointeur



Exemple :

```
Char* p_char = 0;  
Int* p_int = NULL;
```



Déclaration d'un pointeur

- Sur codeblocks
- Reprendre l'exemple précédant et déclarer des pointeurs sur les variables déjà déclarées

Utilisation des pointeurs

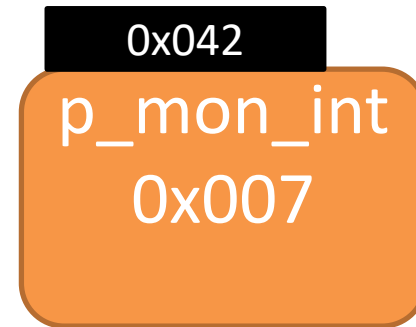


`mon_int` (contenue de `mon_int`)

-> 12

`&mon_int` (adresse de `mon_int`)

-> 0x007



`&p_mon_int` (adresse de `p_mon_int`)

-> 0x042

`p_mon_int` (contenue de `p_mon_int`)

-> 0x007

`*p_mon_int` (contenue de 0x007)

-> 12

- `int main()`
- `{`
- `//printf("Hello world!\n");`
- `int variable =5;`
- `int* p_variable =&variable;`
- `printf("\nle contenu de la variable variable : %d",variable);`
- `printf("\nl adresse de la variable variable : %p",&variable);`
- `printf("\nle contenu de la variable p_variable : %p",p_variable);`
- `printf("\nle contenu de la variable pointee par p_variable : %d",*p_variable);`
- `printf("\nl adresse de la variable p_variable : %p",&p_variable);`
- `return 0;`
- `}`

```
le contenu de la variable variable : 5
l adresse de la variable variable : 000000000061FE1C
le contenu de la variable p_variable : 000000000061FE1C
le contenu de la variable pointee par p_variable : 5
l adresse de la variable p_variable : 000000000061FE10
```

Dangers des pointeurs

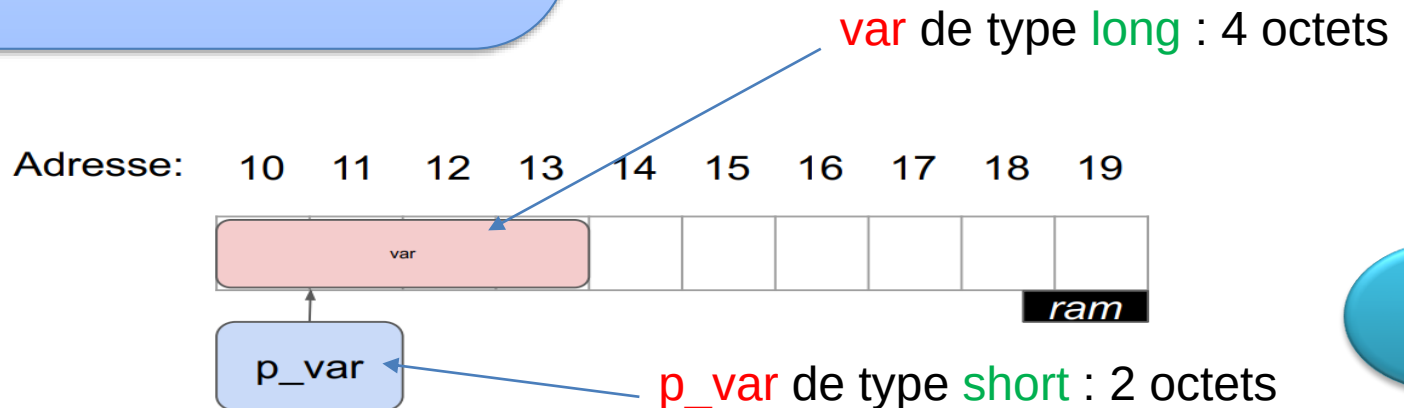
1. Problème de typage :

Les pointeurs doivent correspondre aux types que l'on souhaite pointer

```
int main()
{
    long var = 65321;
    short* p_var = &var;

    printf("ma variable = %ld \n", var);
    printf("ma variable = %ld \n", *p_var);
}
```

ma variable = 65321
ma variable = -215



Dangers des pointeurs

2. Pointeur NULL:

On essaie d'afficher le contenu des variables pointées par NULL

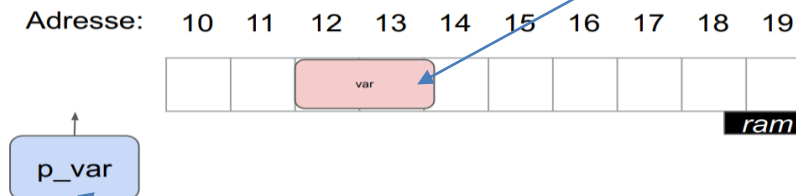
```
int main()
{
    short var = 123;
    short* p_var = NULL; // short* p_var = 0;

    printf("ma variable = %ld \n", var);
    printf("ma variable = %ld \n", *p_var);
}
```

ma variable = 123

Crash !

var de type short : 2 octets



p_var de type short : ne point null part

Dangers des pointeurs

2. Pointeur NULL:

On essaie d'afficher le contenu des variables pointées par NULL

Pour que le programme ne crashe pas :

```
int main()
{
    short var =123;
    short* p_var =NULL; // short* p_var = 0;
    printf("ma variable = %ld \n",var);
    if(p_var ==NULL)
    {
        printf("\n pointeur NULL !!!\n ");
    }
    else
    {
        printf("ma variable = %ld \n",*p_var);
    }
    return 0;
}
```

ma variable = 123

pointeur NULL !!!

Dangers des pointeurs

2. Pointeur NULL:

On essaie d'afficher le contenu des variables pointées par NULL

Pour que le programme ne crashe pas :

```
int main()
{
    short var =123;
    short* p_var =&var;
    printf("ma variable = %ld \n",var);
    if(p_var ==NULL)
    {
        printf("\n pointeur NULL !!!\n ");
    }
    else
    {
        printf("ma variable = %ld \n",*p_var);
    }
    return 0;
}
```

ma variable = 123
ma variable = 123

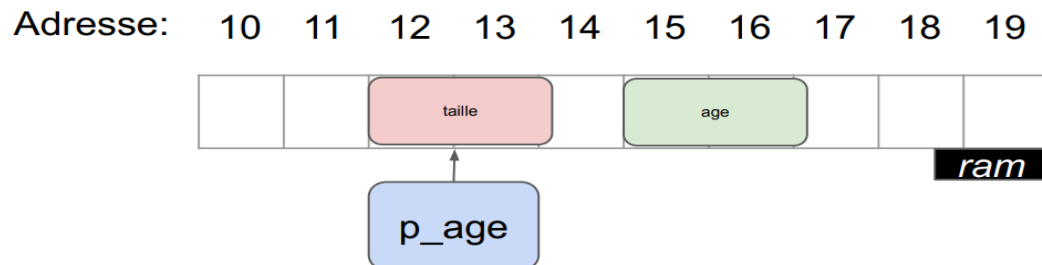
Dangers des pointeurs

3. Jardinage :

On essaie de pointer sur une autre variable valide , problème difficile à trouver !!!

```
int main()
{
    int age=25;
    int taille =165;
    int* p_int =0;
    p_int=&taille;
    printf("\n donner votre age : ");
    scanf("%d",p_int);
    printf("votre age est : %d ans ",age);
    return 0;
}
```

donner votre age : 30
votre age est : 25 ans



Tester nos connaissances

- Jeux kahoot
- <https://create.kahoot.it/details/pointeur/823599f7-6485-4658-825d-523c3bec364c>

EXERCICE

1. Créer une variable de type char
2. Afficher le contenu de la variable, sa taille en mémoire et son adresse

```
Informations sur ma variable:  
type: char  
taille: 1 octet  
contenu: A  
adresse: 0060FF0F
```

Aide : pour afficher une adresse il faut utiliser

%p

- `int main()`
- `{`
- `char lettre = 'A';`
- `printf("\n Informations sur ma vaiabale : \n ");`
- `printf("\t type : char \n ");`
- `printf("\t taille : %d \n ", sizeof(lettre));`
- `printf("\t contenu : %c \n ",lettre);`
- `printf("\t adresse : %p \n",&lettre );`
- `return 0;`
- `}`

```
Informations sur ma vaiabale :
    type       : char
    taille     : 1
    contenu    : A
    adresse    : 0019FF3B

Process returned 0 (0x0)   execution time : 0.206 s
Press any key to continue.
```

FIN

La Programmation en langage C

Chapitre 5: les Tableaux

La cible : MIP S3

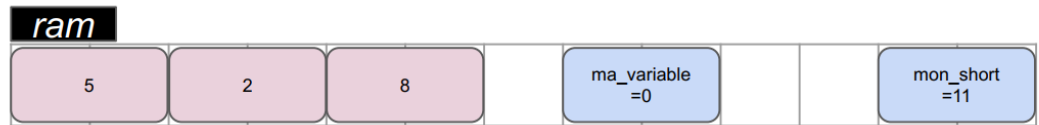
Pr S.ELFILALI

sanaa.elfilali@univh2c.ma

Les tableaux

tableau de 3 cases:

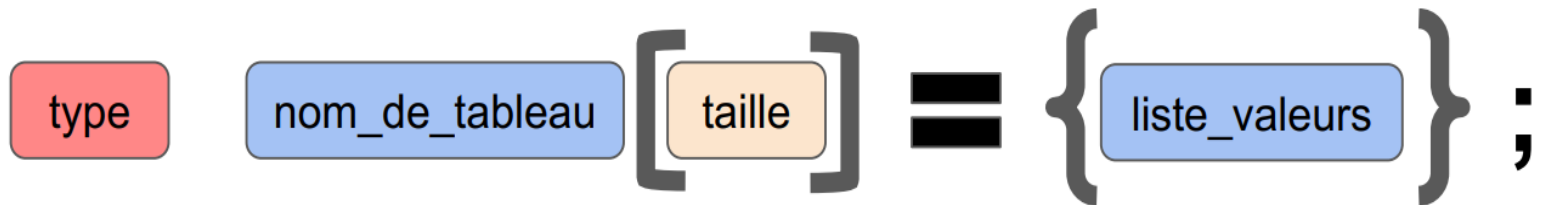
5	2	8
---	---	---



Toutes les cases d'un tableau sont contiguës en mémoire

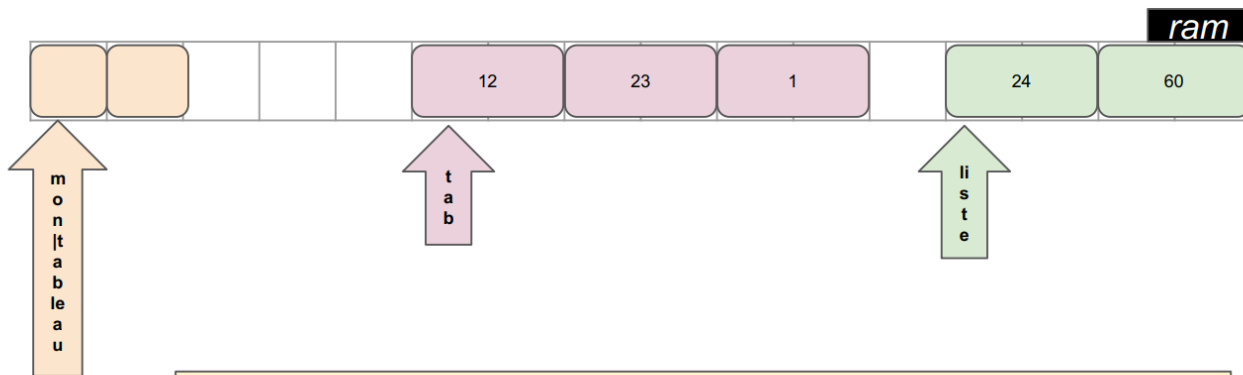
Toutes les cases d'un tableau sont du même type

Déclaration d'un tableau



Exemple:

```
char mon_tableau[2];  
short tab[3] = {12, 23, 1};  
short liste[] = {24, 60}
```



Taille du tableau fixé à la déclaration et ne peut pas varier

Déclaration d'un tableau

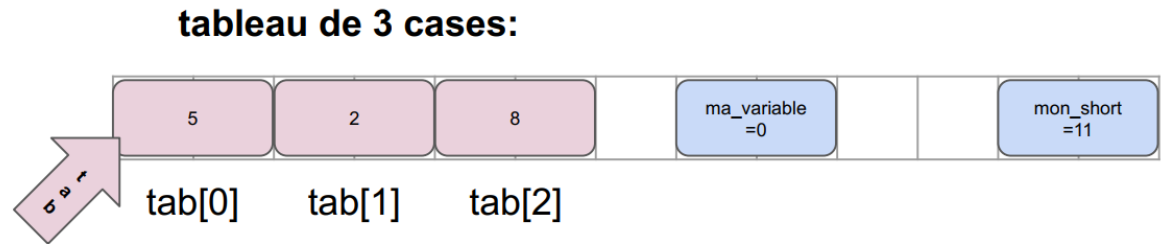
Exemple sur codeblocks

```
int main()
{
    int tirage_loto_boules[5] = {1,2,3,4,5};
    int tirage_loto_etoiles[] = {6,7};
    char initiales_gagnant[2];
    return 0;

}
```

Parcourir un tableau

ID:	0	1	2
	5	2	8



1ere case d'une tableau a pour ID 0

Toutes les cases d'un tableau sont du même type

Toutes les cases d'un tableau sont contiguës en mémoire

Parcourir un tableau

Exemple sur codeblocks

Initialisation du
tableau sans boucle

```
# define Taille_Tableau 3
int main()
{
//declarer le tableau
    int i;
    int mon_tableau[Taille_Tableau];
// init du tableau
    mon_tableau [0]= 0;
    mon_tableau [1]= 1;
    mon_tableau [2]= 2;

// affichage du tableau
    for(i=0;i<Taille_Tableau;i++)
    {
        printf("mon_tableau[%d] =%d \n",i, mon_tableau[i] );
    }
    return 0;
}
```

Initialisation du
tableau avec la boucle

```
#include <stdio.h>

# define Taille_Tableau 3
int main()
{
//declarer le tableau
    int i;
    int mon_tableau[Taille_Tableau];
// init du tableau
    for(i=0;i<Taille_Tableau;i++)
    {
        mon_tableau[i]=i;
    }
// affichage du tableau
    for(i=0;i<Taille_Tableau;i++)
    {
        printf("mon_tableau[%d] =%d \n",i, mon_tableau[i] );
    }
    return 0;
}
```

Parcourir un tableau

Exemple sur codeblocks

```
# define Taille_Tableau 3
int main()
{
//declarer le tableau
    int i;
    int mon_tableau[Taille_Tableau];
// lecture du tableau : la saisie
    for(i=0;i<Taille_Tableau;i++)
    {
        printf("> ");
        scanf("%d",&mon_tableau[i]);
    }
// affichage du tableau
    for(i=0;i<Taille_Tableau;i++)
    {
        printf("mon_tableau[%d] =%d \n",i, mon_tableau[i] );
    }
    return 0;
}
```

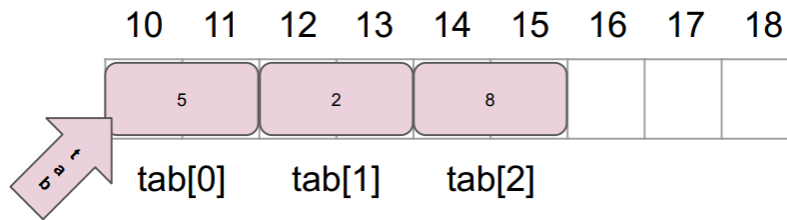


La lecture du tableau
avec la boucle

```
> 12
> 56
> 25
mon_tableau[0] =12
mon_tableau[1] =56
mon_tableau[2] =25

Process returned 0 (0x0)   execution time : 8.344 s
Press any key to continue.
```

Tableaux et pointeurs



`tab` (adresse du tableau)

`*tab` (contenue de la première case du tableau)

`*(tab+n-1)` (contenue de la la case `n` du tableau)

Exemple:

`tab[0] -> 5     // *(tab+0)->5`

`tab[1] -> 2     // *(tab+1)->2`

`tab[2] -> 8     // *(tab+2)->8`

`*tab + 1 => 5 + 1 => 6`

Tableaux et pointeurs

Exemple sur codeblocks

```
# define Taille_Tableau 3
int main()
{
//declarer le tableau
    int i;
    int mon_tableau[Taille_Tableau];
// lecture du tableau : la saisie
    for(i=0;i<Taille_Tableau;i++)
    {
        printf("> ");
        scanf("%d", (mon_tableau+i));
    }
// affichage du tableau
    for(i=0;i<Taille_Tableau;i++)
    {
        printf("mon_tableau[%d] =%d \n", i, *(mon_tableau+i));
    }
    return 0;
}
```

```
> 25
> 14
> 19
mon_tableau[0] =25
mon_tableau[1] =14
mon_tableau[2] =19
```

Tableaux et adresses : Exercice

- 1- Créez un tableau de 5 caractères et initialisez le
- 2- Pour chaque cases, afficher l'index, le contenu et d'adresse
- 3- Faire la même chose mais en utilisant la syntaxe par pointeur

Aide :

```
tab[0] = C (0060FEF9)
tab[1] = O (0060FEFA)
tab[2] = D (0060FEFB)
tab[3] = E (0060FEFC)
tab[4] = R (0060FEFD)
```

- 1- `type nom[taille] = {liste_valeurs};`
- 2- Il faut utiliser `%p` pour afficher une adresse mémoire dans un `printf`.
- 3- `tab[i] = *(tab+i)`

Tableaux et adresses : Exercice

Annotation simple

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    char tab[] = {'C','O','D','E','R'};
    int i;
    // annotation simple

    for(i=0;i<5;i++)
    {
        printf("tab[%d] = %c (%p)\n", i, tab[i], &tab[i]);
    }
    return 0;
}
```

```
tab[0] = C (0019FF34)
tab[1] = O (0019FF35)
tab[2] = D (0019FF36)
tab[3] = E (0019FF37)
tab[4] = R (0019FF38)
```

Tableaux et adresses : Exercice

Annotation pointeur

```
#include <stdio.h>
#include <stdlib.h>

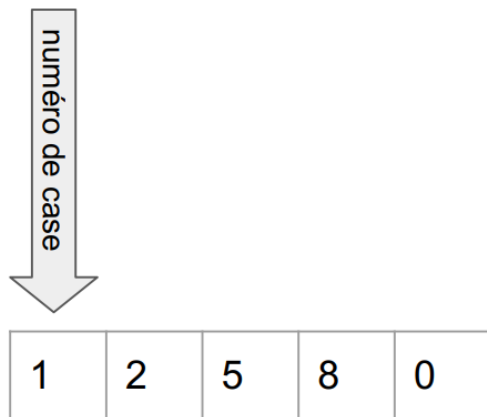
int main()
{
    char tab[] = {'C','O','D','E','R'};
    int i;

    // Annotation Pointeur
    for(i=0;i<5;i++)
    {
        printf("(tab + %d) = %c (%p)\n", i,*(tab+i),(tab+i));
    }

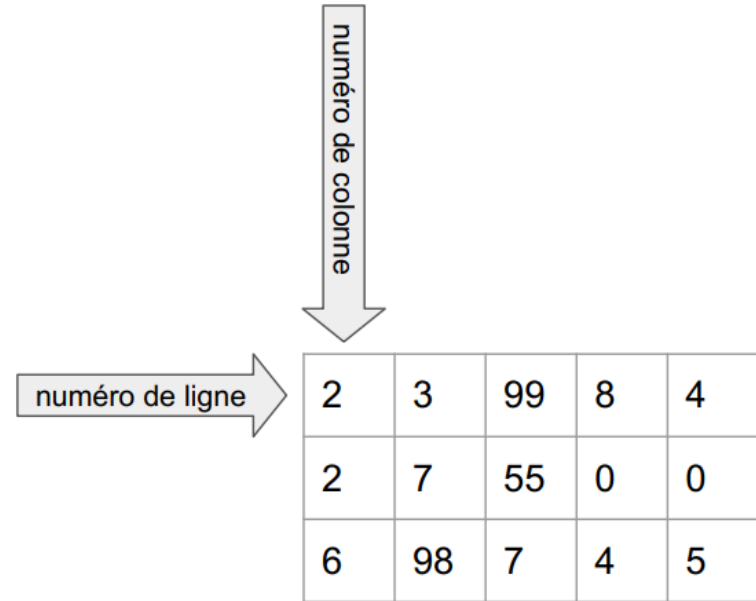
    return 0;
}
```

```
*(tab + 0) = C (0019FF34)
*(tab + 1) = O (0019FF35)
*(tab + 2) = D (0019FF36)
*(tab + 3) = E (0019FF37)
*(tab + 4) = R (0019FF38)
```

Les tableaux multidimensionnels

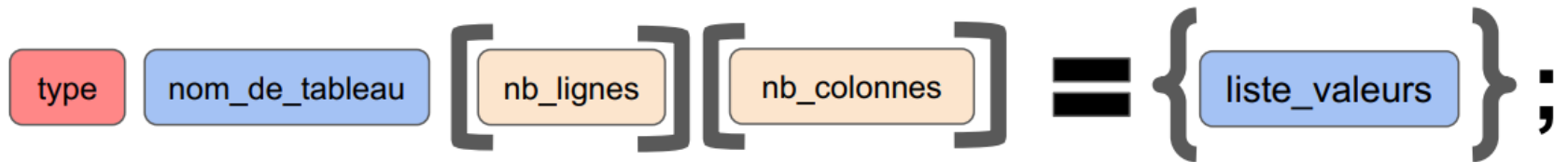


1D



2D

Déclaration tableaux 2D

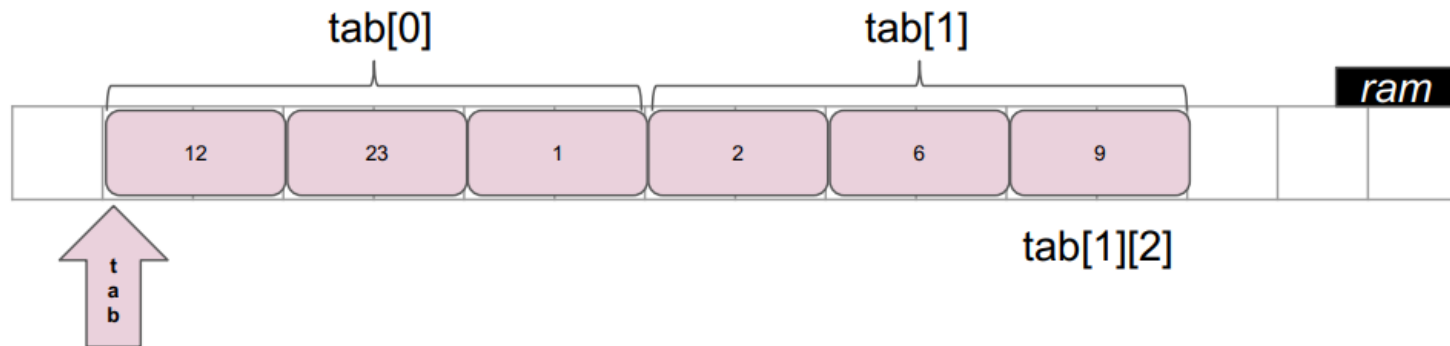


Exemple:

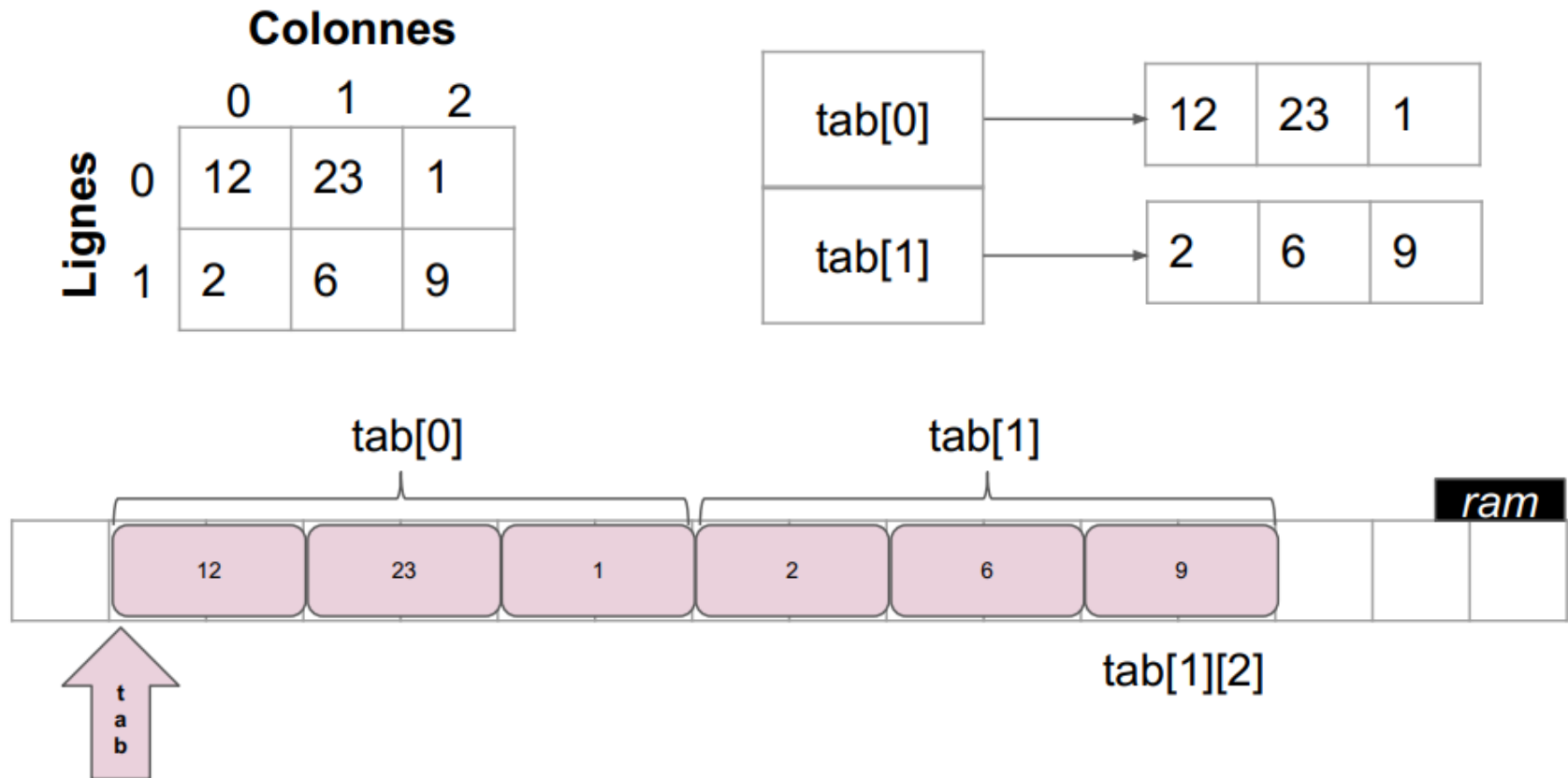
```
short tab[2][3] = { {12, 23, 1}, {2, 6, 9} };
```

```
tab[1][2] //retourne la valeur 9
```

		Colonnes		
		0	1	2
Lignes	0	12	23	1
	1	2	6	9



Tableaux 2D = tableau de tableaux



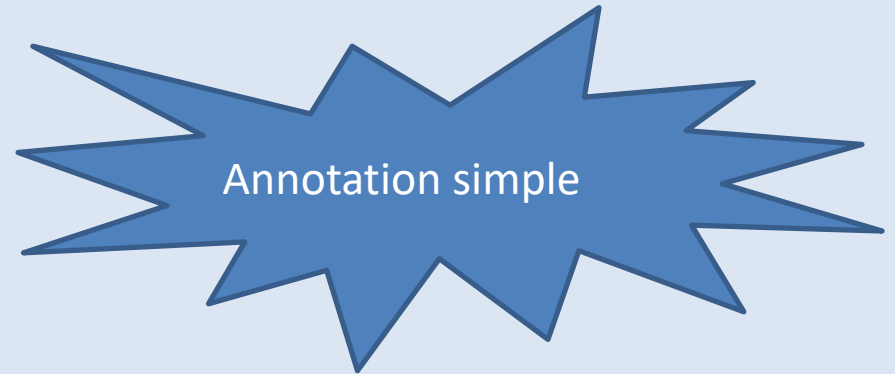
Les tableaux multidimensionnels

Exercices

Exemple sur codeblocks : créer un tableau à deux dimensions

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int ligne;
    int colonne;
    int tab[2][3]={{1,2,3},{4,5,6}};
    //Annotation simple
    for(ligne=0;ligne<2;ligne++)
    {
        for (colonne=0;colonne<3;colonne++)
        {
            printf("tab[%d][%d] = %d\n",ligne,colonne,tab[ligne][colonne]);
        }
    }
    return 0;
}
```



Les tableaux multidimensionnels

Exercices

Exemple sur codeblocks : créer un tableau à deux dimensions

```
#include <stdio.h>
#include <stdlib.h>
```

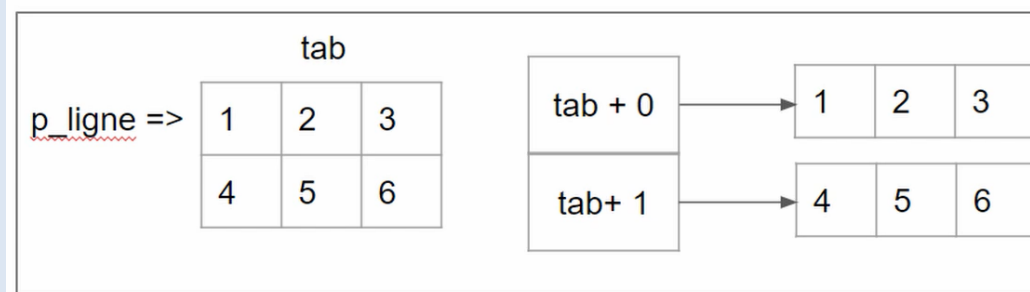
```
int main()
```

```
{
    int ligne;
    int colonne;
    int* p_ligne;
    int tab[2][3]={1,2,3},{4,5,6}};
    for(ligne=0;ligne<2;ligne++)
    {
```

```
        p_ligne = *(tab+ligne);
        for (colonne=0;colonne<3;colonne++)
        {
            printf("tab[%d][%d] = %d\n",ligne,colonne,*(p_ligne +colonne));
        }
    }
    return 0;
}
```

Annotation pointeur

```
tab[0][0] = 1
tab[0][1] = 2
tab[0][2] = 3
tab[1][0] = 4
tab[1][1] = 5
tab[1][2] = 6
```



`*(*(tab+ligne)) = *(p_ligne)`

Mini projet

Travaux Pratiques Programmation en langage C

SMI_ S3

TP : les Tableaux
MASTERMIND



Pr S.ELFIALI
2020-2021

Mini projet

Nous allons créer un petit jeu de type mastermind. Pour ceux qui ne connaissent pas, ce jeu de société se joue à deux, une des personnes crée un code secret composé de 4 couleurs. Puis le 2em joueur doit découvrir cette combinaison. Vous pouvez retrouver les règles du jeu ici: <https://www.regles-de-jeux.com/regle-du-mastermind/>

Nous allons simplifier un peu les règles pour ce TP.

- Nous allons utiliser les initiales des couleurs pour lire la saisie utilisateur et stocker le code secret parmi ('R','V','B','J','O')
- Le code sera constitué de 4 couleurs.
- Le code ne sera pas choisi par un joueur mais écrit en dure dans le programme (ex {'R','V','B','J'})
- Le nombre maximal de tentatives via une constante (ex: 3)

```
...  
Donnez un code de 4 couleurs differentes et sans espaces parmi : {'R', 'V', 'B', 'J', 'O'}  
> RVJO  
Tentative 1/3  
Couleurs mal placees: 1  
Couleurs bien placees: 2
```

Application : Le Tri

La Programmation en langage C

Chapitre 7 : les chaînes de caractères

La cible : MIP S3

Pr S.ELFILALI
sanaa.elfilali@univh2c.ma

les chaines de caractères

Que signifie une chaine de caractères ?

Une suite logique de caractères et qui vont constituer des mots ou des phrases

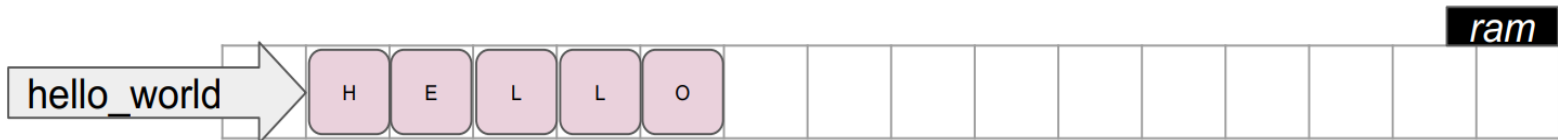
NB : En général en programmation il y a le type string qui permet de stocker des chaines de caractères mais en langage C , ce type de variable n'est pas disponible ! Mais on peut manipuler ou utiliser le string en C

Les tableaux de char

Exemple:

```
char hello_world[5] = { 'H', 'E', 'L', 'L', 'O' };
```

H	E	L	L	O
---	---	---	---	---



Les tableaux de char

Illustration sur CodeBlocks

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int i;
    char salut[]={'S','A','L','U','T'};
    for(i=0;i<5;i++)
    {
        printf("%c",salut[i]);
    }

    return 0;
}
```

```
SALUT
Process returned 0 (0x0)   execution time : 0.026 s
Press any key to continue.
```


Les strings

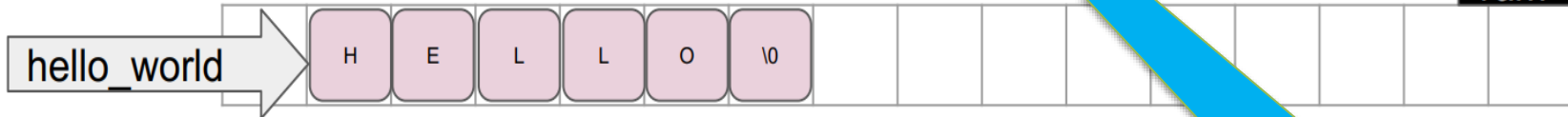
Exemple:

```
char hello_world[6] = { 'H', 'E', 'L', 'L', 'O', '\0' };  
printf("%s", hello_world) -> HELLO
```

Pas besoin de boucle !!!
Juste %s

Un marqueur
Un caractère spécial

H	E	L	L	O	\0
---	---	---	---	---	----



La fin de la chaîne
Pas la fin du tableau !

Les strings

Illustration sur CodeBlocks

```
int main()
{
    int i;
    char salut[]={'S','A','L','U','T','\0'};

    printf("%s",salut);

    return 0;
}
```

```
int main()
{
    int i;
    char salut[]={'S','A','L','\0','U','T','\0'};

    printf("%s",salut);

    return 0;
}
```

Si on met \0 ici

```
SALUT
Process returned 0 (0x0)   execution time : 0.026 s
Press any key to continue.
```

```
SAL
Process returned 0 (0x0)   execution time : 0.078 s
Press any key to continue.
```

Les strings

Illustration sur CodeBlocks

```
int main()
{
    int i;
    char salut[] = "SALUT";
    printf("%s",salut);

    return 0;
}
```

Initialisation avec une chaîne de caractère directement

```
SALUT
Process returned 0 (0x0)   execution time : 0.026 s
Press any key to continue.
```

```
int main()
{
    int i;
    char salut[]="SALUT";
    char prenom[100];
    scanf("%s", prenom);
    printf("%s, %s",salut, prenom);
    return 0;
}
```

```
Sanaa
SALUT, Sanaa
Process returned 0 (0x0)   execution time : 7.637 s
Press any key to continue.
```

Si vous mettez un prénom composé, il prend en considération que le premier mot!! (l'espace est une validation comme entrée

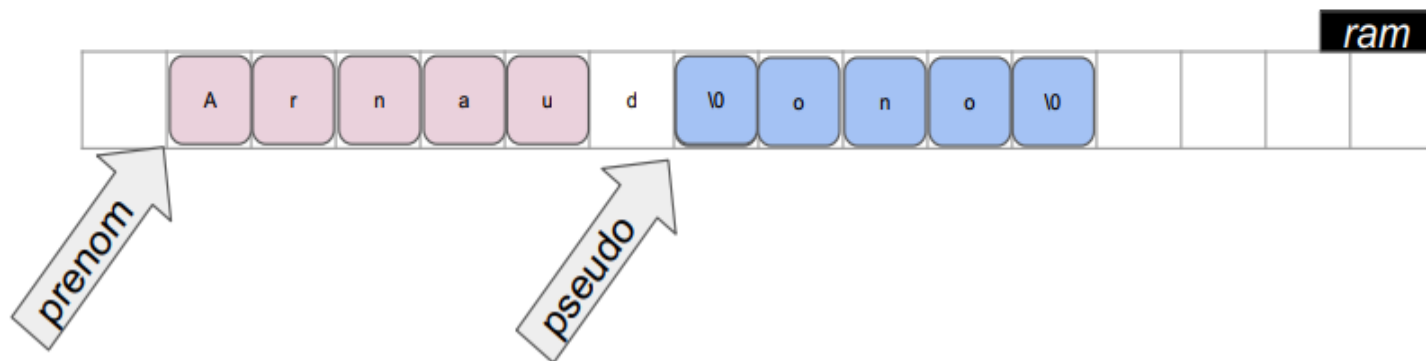
Dépassement de mémoire

Exemple:

```
char pseudo[5];  
pseudo = "Nono";
```

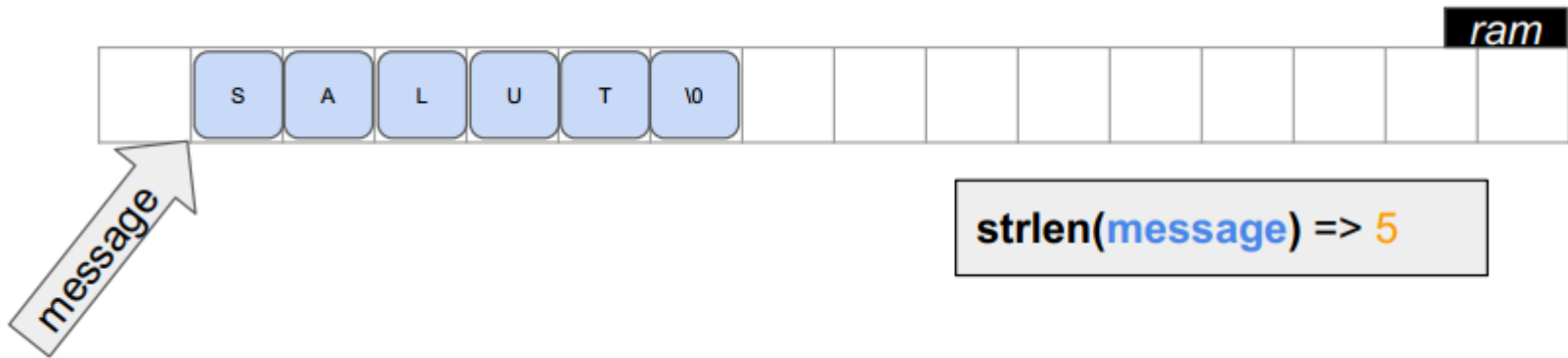
```
char prenom[5];  
prenom = "Arnaud";
```

```
printf("%s", prenom); => Arnaud  
printf("%s", pseudo); =>
```



Strlen(string.h)

taille strlen(chaîne)



[C documentation — DevDocs](https://devdocs.io/c/) : <https://devdocs.io/c/>

Retourne la taille de la string passée en paramètre en excluant le ' '

Strlen(string.h)

Illustration sur CodeBlocks

```
include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

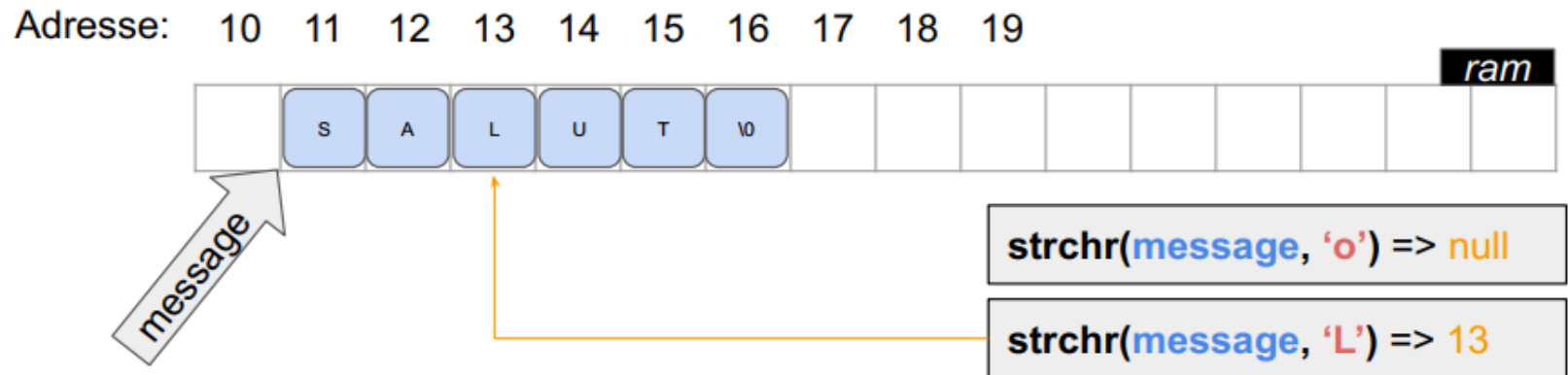
```
int main()
{
    char message[] = "hello world";
    printf("taille de message \" %s \" = %d \n",message, strlen(message));

    return 0;
}
```

taille de message " hello world " = 11

Strchr(string.h)

pointeur strchr(chaîne , char)



Retourne la première sous chaîne qui commence par le caractère passé en argument.

Si le caractère n'est pas trouvé, elle retourne NULL.

Strchr(string.h)

Illustration sur CodeBlocks

```
int main()
{
    char message[] = "hello world";
    char char_a_trouver = 'v';
    char* p_char = strchr(message ,char_a_trouver);

    printf("taille de message \" %s \" = %d \n",message, strlen(message));

    if (p_char ==NULL)
    {
        printf("la char \'%c\' n'est pas present dans la chaine \" %s \"",char_a_trouver,message);
    }
    else
    {
        printf("la char \'%c\'est present dans la chaine \" %s \" ",char_a_trouver,message)
    }
    return 0;
}
```

taille de message " hello world " = 11
la char 'v' n'est pas present dans la chaine " hello world "

Strchr(string.h)

Illustration sur CodeBlocks

```
int main()
{
    char message[] = "hello world";
    char char_a_trouver = 'e';
    char* p_char = strchr(message ,char_a_trouver);

    printf("taille de message \" %s \" = %d \n",message, strlen(message));

    if (p_char ==NULL)
    {
        printf("la char \'%c\' n'est pas present dans la chaine \" %s \"",char_a_trouver,message);
    }
    else
    {
        printf("la char \'%c\'est present dans la chaine \" %s \" ",char_a_trouver,message);
    }
    return 0;
}
```

taille de message " hello world " = 11
la char 'e'est present dans la chaine " hello world "

Strchr(string.h)

Illustration sur CodeBlocks

```
int main()
{
    char message[] = "hello world";
    char char_a_trouver = 'e';
    char* p_char = strchr(message + 3, char_a_trouver);

    printf("taille de message \" %s \" = %d \n", message, strlen(message));

    if (p_char == NULL)
    {
        printf("la char \" %c \" n'est pas present dans la chaine \" %s \"", char_a_trouver, message);
    }
    else
    {
        printf("la char \" %c \" est present dans la chaine \" %s \" ", char_a_trouver, message);
    }
    return 0;
}
```

taille de message " hello world " = 11
la char 'e' n'est pas present dans la chaine " hello world "

La recherche commence à partir du troisième caractère pour faire la recherche dans un sous ensemble de chaine

Strchr(string.h)

Illustration sur CodeBlocks

```
int main()
{
    char message[] = "hello world";
    char char_a_trouver = 'o';
    char* p_char = strchr(message + 3, char_a_trouver);

    printf("taille de message \" %s \" = %d \n", message, strlen(message));

    if (p_char == NULL)
    {
        printf("la char \" %c \" n'est pas present dans la chaine \" %s \" ", char_a_trouver, message);
    }
    else
    {
        printf("la char \" %c \" est present dans la chaine \" %s \" la fin du message apres le char est %s", char_a_trouver, message, p_char);
    }
    return 0;
}
```

taille de message " hello world " = 11
la char 'o' est present dans la chaine " hello world " la fin du message apres le char est o world

Il retourne le premier
caractère trouvé si il y a des
doublons

Seance suivante

Strcmp(string.h)

`int strcmp( chaîne 1, chaîne 2 )`

`strcmp("Tata", "Toto") => -1`

`strcmp("Toto", "Toto") => 0`

`strcmp("Toto", "Tata") => 1`

Compare deux chaînes, retourne 0 si elles sont identiques, sinon elle retourne -1 dans le cas où la première chaîne est avant dans l'ordre alphabétique et elle retourne 1 dans l'autre cas.

Strcmp(string.h)

Illustration sur CodeBlocks

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main()
{
    char message[] = "hello world";

    if(strcmp(message,"hello world")== 0)
    {
        printf("\nles deux chaines sont identiques\n");
    }
    else
    {
        printf("\nles deux chaines sont differentes \n");
    }
    return 0;
}
```

les deux chaines sont identiques

Strcmp(string.h)

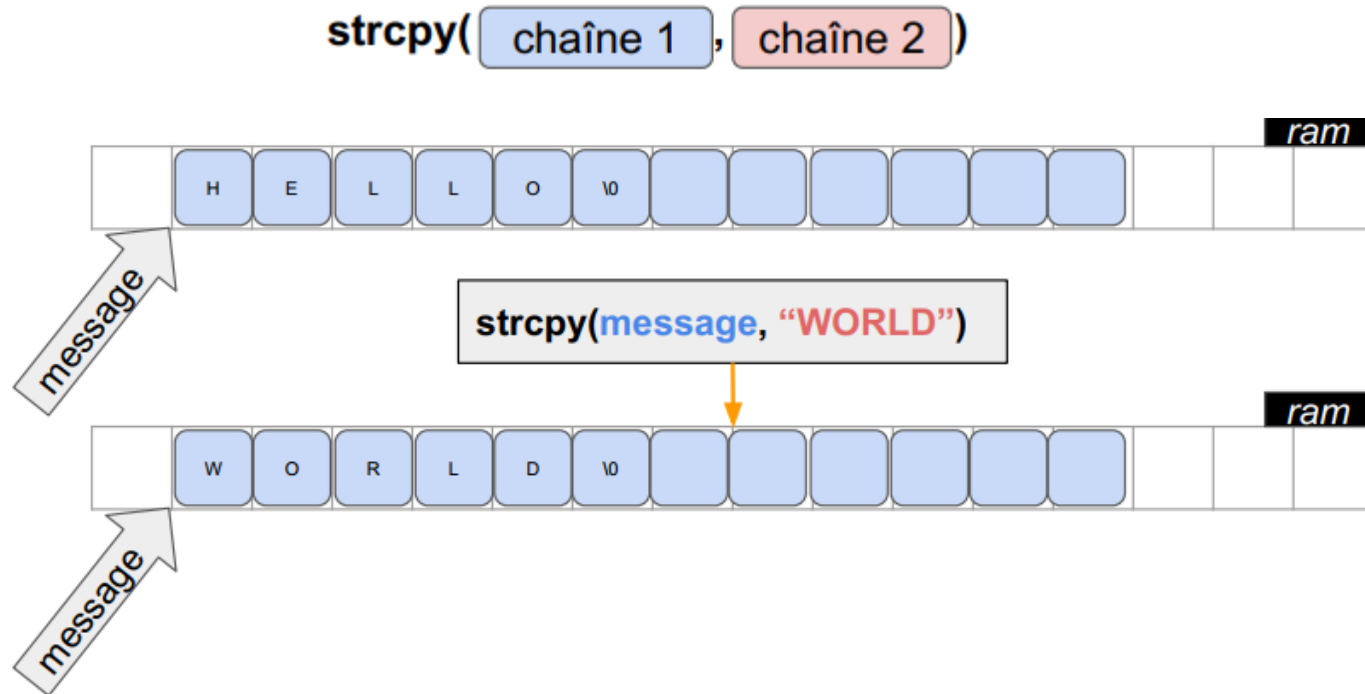
Illustration sur CodeBlocks

```
int main()
{
    char message[] = "hello world";

    if(strcmp(message,"hello SMI")== 0)
    {
        printf("\nles deux chaines sont identiques\n");
    }
    else
    {
        printf("\nles deux chaines sont differentes \n");
    }
    return 0;
}
```

les deux chaines sont differentes

Strcpy(string.h)



**Copie le contenu de la chaîne 2 dans la chaîne 1.
Attention à ce que le tableau de chaîne 1 soit suffisamment grand pour
contenir chaîne 2.**

Strcpy(string.h)

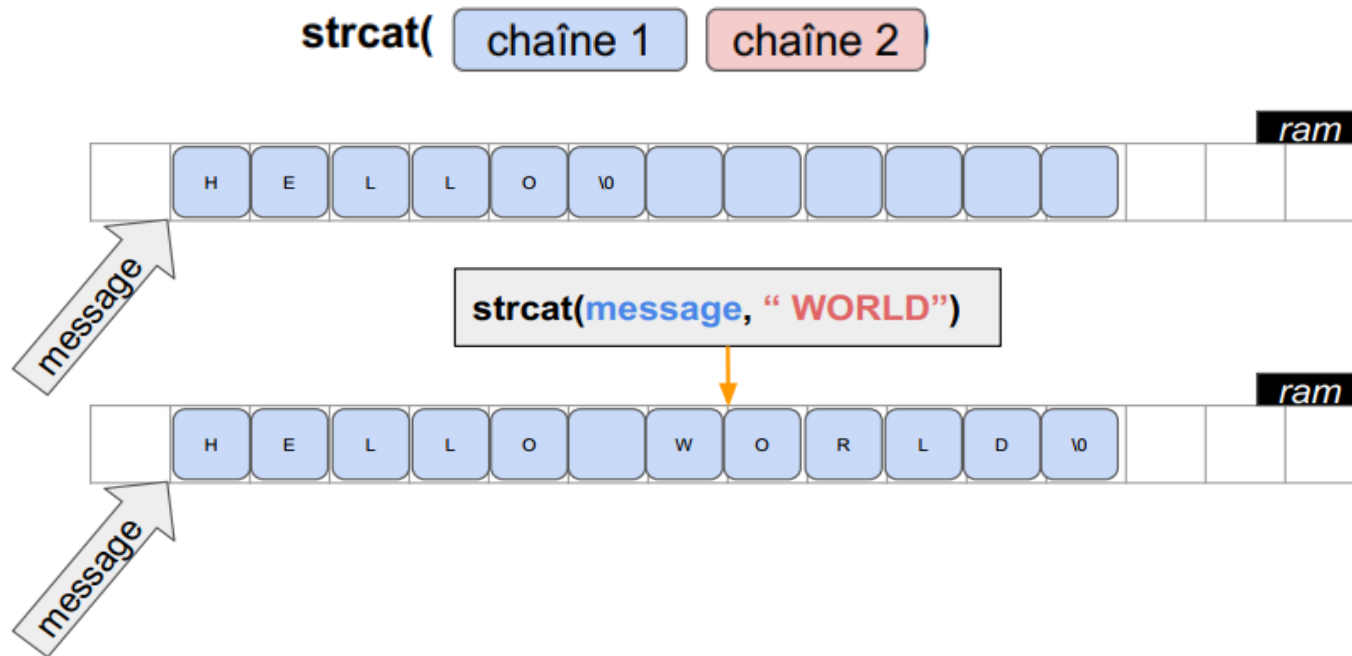
Illustration sur CodeBlocks

```
int main()
{
    char message[100] = "hello";

    printf("message = %s\n",message);
    strcpy(message,"world");
    printf("message = %s \n",message);
    return 0;
}
```

```
message = hello
message = world
```

Strcat(string.h)



**Insère le contenu de la chaîne 2 devant la chaîne 1.
Attention à ce que le tableau de chaîne 1 soit suffisamment grand pour
contenir chaîne 1 et chaîne 2.**

Strcat(string.h)

Illustration sur CodeBlocks

```
int main()
{
    char message[100] = "hello";

    strcat(message, " world");
    printf("message = %s \n", message );
    return 0;
}
```

message = hello world

Strtol(string.h)

`long` `strtol`(`chaîne` , `pt_fin` , `base`)

`strtol("123456", NULL, 10) => 123456`

`strtol("12€", NULL, 10) => 12`

Convertit une chaîne en valeur numérique entière qu'elle représente.

pt_fin : la fonction s'en sert pour renvoyer la position du premier caractère qu'elle a lu et qui n'était pas un nombre. On peut mettre NULL si on ne souhaite pas l'utiliser.

base : base avec laquelle le nombre est écrit dans la chaîne. Le plus souvent base 10

Strtod(string.h)

`double strtod( chaîne , pt_fin )`

`strtod("12.3", NULL) => 12.3`

`strtod("12.2€", NULL) => 12.2`

Convertit une chaîne en valeur numérique flottante qu'elle représente.

pt_fin : la fonction s'en sert pour renvoyer la position du premier caractère qu'elle a lu et qui n'était pas un nombre. On peut mettre NULL si on ne souhaite pas l'utiliser.

Strtol(string.h) et Strtod(string.h)

Illustration sur CodeBlocks

```
int main()
{
    char message[100] = "hello";
```

```
    printf("l entier de %s = %d \n ", "1234", strtol("1234", NULL, 10));
    printf("le float de %s = %f \n ", "123,4", strtod("123,4", NULL));
```

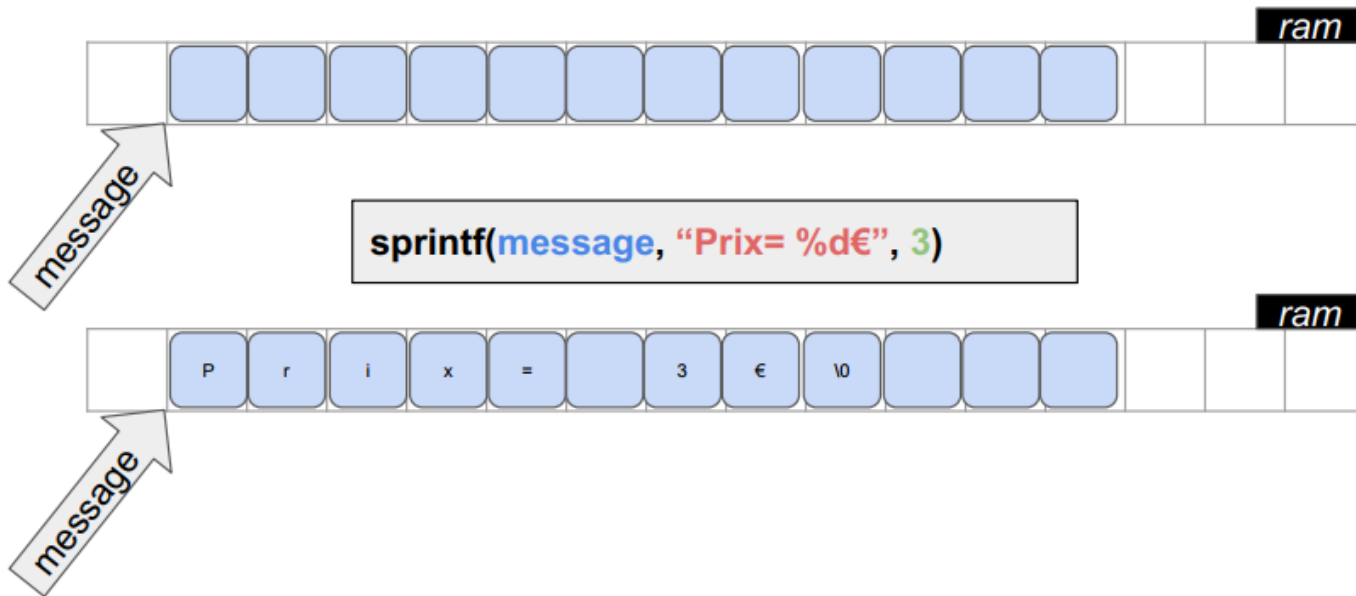
```
    return 0;
}
```

```
l entier de 1234 = 1234
le float de 123,4 = 123.000000
```

A suivre

Sprintf(stdio.h)

sprintf(chaîne destination , chaîne template , params)



Comme la fonction `printf`, `sprintf` permet de formater une chaîne de caractère mais au lieu de l'afficher dans la console, on l'écrit dans un tableau de char (string).

Sprintf(stdio.h)

Illustration sur CodeBlocks

```
int main()
{
    char message[100] = "hello";
    char Nom[20];
    int Age;
    sprintf(message,"mon nom est %s et j ai %d ans ","SANAA",52);
    printf("%s",message);
    return 0;
}
```

mon nom est SANAA et j ai 52 ans

Exercice : Carte d'identité

Objectifs :

1- Créer un programme qui demande à l'utilisateur les informations suivantes et les affiche:

- Date de naissance: string au format jj/mm/aaaa
- Nom: string
- Prenom: string
- Couleur des yeux: string

donnez les informations suivantes :

Date de naissance jj/mm/aaaa :24/05/2002

Nom : KENZA

Prenom :TLEMCANI

Couleur des yeux :VERTS

vous vous appelez KENZA TLEMCANI , vous etes nee le 24/05/2002 et vos yeux sont VERTS

Aide

Il faut utiliser %s dans un scanf pour lire une string. Attention, un espace dans la saisie est considéré comme un séparateur de paramètres dans scanf.

Exercice : Carte d'identité

```
int main()
{
    char date[100];
    char nom[100];
    char prenom[100];
    char couleur[100];
    // lecture saisie utilisateur
    printf("donnez les informations suivantes : \n");
    printf("\t Date de naissance jj/mm/aaaa :");
    scanf("%s",date);
    printf("\t Nom :");
    scanf("%s",nom);
    printf("\t Prenom :");
    scanf("%s",prenom);
    printf("\t Couleur des yeux :");
    scanf("%s",couleur);
    // affichage des informations
    printf("vous vous appelez %s %s , vous etes nee le %s et vos yeux sont %s",nom,prenom,date,couleur);

    return 0;
}
```

donnez les informations suivantes :

Date de naissance jj/mm/aaaa :24/05/2002

Nom : KENZA

Prenom :TLEMCANI

Couleur des yeux :VERTS

vous vous appelez KENZA TLEMCANI , vous etes nee le 24/05/2002 et vos yeux sont VERTS

Refaire l'exercice avec sprintf

lecture sécurisée avec fgets

retour fgets(chaîne , taille , stdin)

```
#include <stdio.h>
#include <stdlib.h>
# define taille 100
int main()
{
    char message[taille];
    printf(« Ecrire un message de %d caracteres maximum \n>",taille);
    if (fgets(message,taille,stdin)!=NULL)
    {
        printf("\n %s \n",message);
    }
    else
    {
        printf("une erreur est survenue \n");
    }

    return 0;
}
```

Ecrire un message de 100 caracteres maximum
>bienvenue au cours de langage de programmation C

bienvenue au cours de langage de programmation C

fgets retourne NULL si il y a une erreur lors de la lecture

lecture sécurisée avec fgets

Illustration sur CodeBlocks

```
int main()
{
    char nom[100];
    printf("comment tu t'appelles ? ");
    scanf("%s",nom);

    printf("tu t'appelles %s ", nom );
    return 0;
}
```

comment tu t'appelles ? sanaa
tu t'appelles sanaa

comment tu t'appelles ? Sanaa EL FILALI
tu t'appelles Sanaa

Les caracteres de validation de la saisie avec scanf

Comment maitriser les caractères ('\\', ' ', ';', ':', '!', ' ') de validation de la saisie avec scanf

lecture sécurisée avec fgets

Illustration sur CodeBlocks

```
int main()
{
    char nom[100];
    printf("comment tu t'appelles ? ");
    if(fgets(nom, 99, stdin) != NULL)
    {
        printf("tu ne sais rien %s ! ", nom);
    }
    return 0;
}
```

comment tu t'appelles ? Sanaa
tu ne sais rien Sanaa
!

```
int main()
{
    char nom[100];
    printf("comment tu t'appelles ? ");
    if(fgets(nom, 5, stdin) != NULL)
    {
        printf("tu ne sais rien %s ! ", nom);
    }
    return 0;
}
```

comment tu t'appelles ? SANAA
tu ne sais rien SANA !

Limiter la saisie

lecture sécurisée avec fgets

Illustration sur CodeBlocks

```
int main()
{
    char prix[100];
    printf("donner un prix ? ");

    if(fgets(prix,99,stdin) != NULL)
    {
        printf("vous avez indique %f \n", strtod(prix,NULL));
    }
    return 0;
}
```

donner un prix ? 368.25
vous avez indique 368.250000

Jeux pour valider les connaissances

<https://create.kahoot.it/share/chaine-de-caractere/e8de39a6-04b4-4ef2-97d0-a39976b125d6>

Devoir à faire un jeu : le pendu

TP : les chaînes de caractères

Le pendu



Devoir à faire un jeu : le pendu

A. Enoncé :

Nous allons créer un petit jeu de type pendu. L'objectif est de deviner un mot caché (ex: -----). Pour cela le joueur peut proposer des lettres. Si une lettre est bien présente dans le mot alors elle est placée sur le mot caché (ex: --LL-). Si le joueur propose une lettre qui ne se trouve pas dans le mot alors il perd une vie.

Nous allons simplifier un peu les règles pour ce TP.

- Le mot à deviné sera écrit en dure dans le code (ex: LOL).
- Pas de mot composé (pas d'espaces).
- Le joueur ne peut que proposer des lettres une par une (pas de proposition du mot entier).
- Le joueur commence avec 10 points de vie.

```
...  
Proposer une lettre > P  
Non la lettre 'P' n'est pas présente dans le mot "---", il vous reste 9 vies  
Proposer une lettre > L  
Oui la lettre 'L' est bien présente dans le mot "L-L"  
Proposer une lettre > O  
Bravos, vous avez trouve le mot "LOL" et il vous reste 9 vies  
...
```

Devoir à faire un jeu : le pendu

A. Préparatifs

1- Créer un nouveau projet C du nom de "tp_les_chaines_de_caracteres ".

A. Instructions

1. Créer les variables et constantes nécessaires au programme (mot_secret, saisie_utilisateur, ...).
2. Créer la boucle de jeu
 - 2.1- Demander au joueur de faire une proposition de lettre (attention, utiliser la commande "fflush(stdin);" avant le scanf pour ne pas risquer de lire des restes entré au clavier).
 - 2.2- Si la proposition est bonne remplacer les '-' correspondants par la lettre. Sinon faire perdre une vie au joueur.
- 3- Gérer la fin de partie. Victoire: Cas où le joueur a trouvé toutes les lettres ou Défaite: cas où le joueur n'a plus de vies

Devoir à faire un jeu : le pendu

A. Pour aller plus loin !

Offrir la possibilité, au joueur, de

soit donner une lettre,

soit proposer directement le mot entier si il pense avoir deviné le bon mot.

Devoir à faire

un jeu : le pendu

De l'Aide..

1- Il faut créer les variables suivantes

- int nb_vie=10, char saisie_utilisateur
- le tableau qui contient la chaîne secrète : char mot_secret[] = "PROGRAMMATION";
- la taille de la chaîne secrète : int taille_mot = strlen(mot_secret);
- le tableau des char trouvés : char mot_trouve[taille_mot+1]; (Attention au + 1 dans la taille du Tableau pour avoir une case pour le '\0')
- Enfin il faut initialiser ce tableau avec des '-' et ne pas oublier le '\0' dans la dernière case

2.1- Utiliser fflush(stdin) puis un simple scanf car ici, il faut lire un simple char (attention au & devant votre variable char dans le scanf)

2.2- Il faut parcourir le tableau contenant le mot secret et vérifier pour chaque case si la saisie utilisateur est identique. Si le char est présent dans le mot secret, on l'ajoute au tableau "mot_trouve" sinon on fait perdre une vie.

3- Il faut faire une boucle do while et vérifier que le nombre de points de vie est supérieur à 0. Pour la victoire, il faut comparer la chaîne trouvée avec la chaîne du mot secret via strcmp. si elles sont identique c'est gagné ! il ne faut pas oublier de forcer la sortie de la boucle de jeu via un break;

A. Il faut remplacer la saisie utilisateur par un tableau de char et utiliser la fonction fgets au lieu du scanf. Ainsi il est possible, pour l'utilisateur, d'écrire soit un char soit un string.

Il faut séparer le cas où l'utilisateur rentre une lettre et le cas où il en rentre plusieurs (proposition d'un mot)

La Programmation en langage C

Chapitre 6: les structures

La cible : MIP S3

Pr S.ELFILALI

sanaa.elfilali@univh2c.ma

les structures

Type Estructure

Aussi appelé

Type Enregistrement

ou

C'est un type de données qui permet de décrire une entité composée

L'entité composée est définie par l'intermédiaire d'un ensemble de descripteurs appelés :

propriétés
ou
champs de l'enregistrement.

qui peuvent être de types différents.

les structures

```
Struct NomDeStructure {
```

liste des champs

```
} liste de variables ;
```

Chaque champ est déclaré avec son type et le nom du champ

```
Typedef Struct {
```

liste des champs

```
} NomDeStructure ;
```

NomDeStructure liste des variables ;

Dans ce cas, **NomDeStructure** devient un nouveau type

les structures

- Exemple

Définition du type enregistrement :

```
typedef struct {  
    int x, y ;  
    int couleur ;  
} Point ;
```

Déclaration des variables :

```
Point p1, p2 ;
```

les structures

Accès aux champs :

Variable :

On utilisera le sélecteur « . » : enregistrement.champ

Exemple :

```
p1.x = 15 ;  
p1.y = 27 ;
```

```
p1.x = 0 ;  
p1.y = 2 ;
```

les structures

Lecture écriture des structures :

Se fait champ par champ

Exemple de lecture

```
printf("entrer X : ") ; scanf("%d", &p.x) ;  
printf("entrer Y : ") ; scanf("%d", &p.y) ;
```

Exemple d'affichage

```
printf("(%d , %d)\n", p.x, p.y) ;
```

Remarque :

Un champ d'un enregistrement peut lui aussi être de type enregistrement :

les structures

Un champ d'un enregistrement peut lui aussi être de type enregistrement

Exemple

```
typedef struct {  
    int j, m, a ;  
} Date ;
```

```
typedef struct {  
    int code ;  
    char nom[20] ;  
    Date dn ;  
    ...  
} Personne ;
```

L'accès à l'information de date complète

```
Personne pers ;  
pers.dn.j = 20 ;  
pers.dn.m = 12 ;  
pers.dn.a = 1990 ;
```

pour lire ces informations on le fait
champ par champ :

```
scanf("%d", &pers.dn.j) ;
```

Remarque :

L'affectation entre enregistrements de même type est possible

Exemple

```
Personne p1, p2;  
p1 = p2
```

Tableau de structures

Supposons que nous avons beaucoup d'informations se rapportant à un même objet et que nous devons traiter plusieurs de ces objets.

```
- int  marque[MAX] ;  
- int  modele[MAX] ;  
- int  cassette[MAX] ;  
- int  cd[MAX] ;  
- int  mp3[MAX] ;  
- float prix[MAX] ;
```

En utilisant un tableau pour chaque information, on obtient 6 tableaux

Tableau de structures

- L'utilisation de tableaux à 2 dimensions peut diminuer un peu le nombre de tableaux à gérer
 - `int marque[MAX];`
 - `int modele[MAX];`
 - `int options[MAX][3];`
 - `float prix[MAX];`
- On obtient ainsi 4 tableaux
- Il serait intéressant de pouvoir contenir toute l'information dans un seul tableau

Tableau de structures

- **Déclaration de la structure**

- `typedef struct{`

- `int marque;`

- `int modele;`

- `int cassette;`

- `int cd;`

- `int mp3;`

- `float prix;`

- `} radio;`

Tableau de structures

Déclaration d'un tableau de structures

```
radio tabRadio[MAX] ;
```

Avantage

Un seul tableau à passer en paramètre à des fonctions

Inconvénient

- On ne peut plus passer un tableau contenant une seule information
- Les fonctions qui doivent faire un traitement sur une information en particulier de la structure doivent gérer l'accès à celle-ci.

FIN

La Programmation en langage C

Chapitre 7 : Les fonctions

La cible : MIP S3

Pr S.ELFILALI
sanaa.elfilali@univh2c.ma

La fonction main et printf

```
include <stdio.h>
```

```
int main()  
{  
    printf("message 1");  
    printf("message 2");  
    ...  
    return 0;  
}
```

```
void printf()  
{  
    .....  
}
```

C'est quoi une fonction ?

Une partie du code (programme), des blocs d'instructions que l'on peut ensuite réutiliser dans le code source en l'appelant par son nom

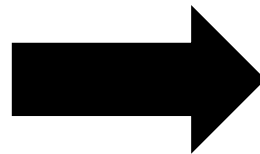
Utilisation des fonctions

```
int main()
{
    printf("Salut !");
    printf("Formation C");

    printf("Salut !");
    printf("Formation C");

    printf("Salut !");
    printf("Formation C");

    return 0;
}
```



```
int main()
{
    maFonction();
    maFonction();
    maFonction();
    return 0;
}
```

```
void maFonction()
{
    printf("Salut !");
    printf("Formation C");
}
```

Pourquoi utiliser les fonctions ?

Intérêt 1 : découper le programme en blocs pour pouvoir le réutiliser par la suite
Ensuite; mutualiser , factoriser pour éviter d'avoir plein de duplications de copier coller dans ce code source

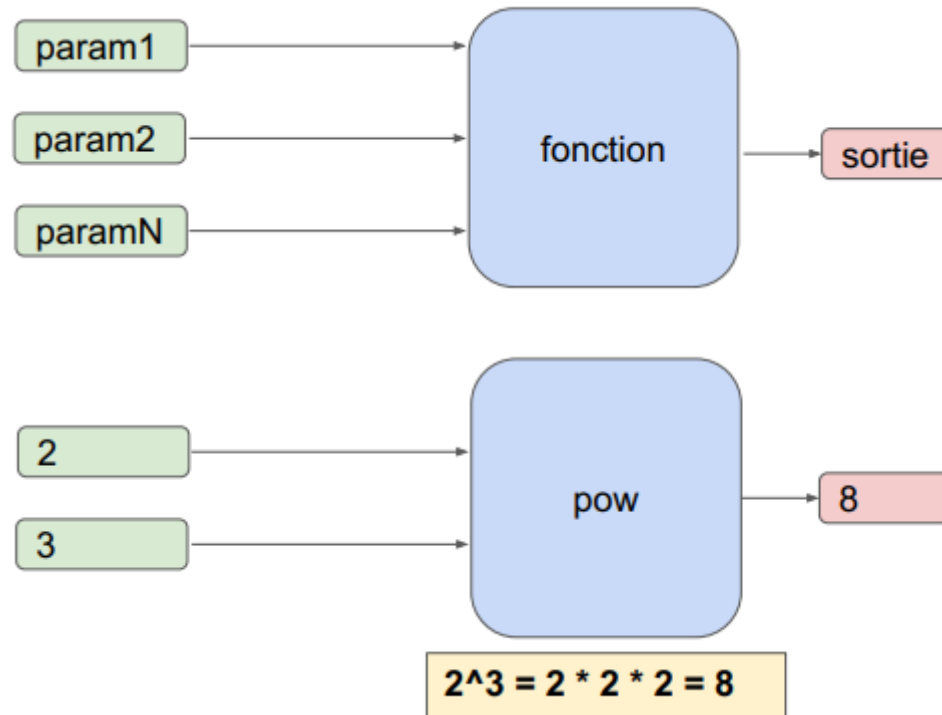
Intérêt 2 : découper le programme de manière logique pour pouvoir le rendre plus facile à comprendre par la suite

Utilisation des fonctions

```
int main()
{
    maFonction();
    maFonction();
    maFonction();
    return 0;
}
```

```
void maFonction()
{
    printf("Salut !");
    printf("Formation C");
}
```

Les fonctions



De quoi est composé une fonction ?

Un bloc d'instruction

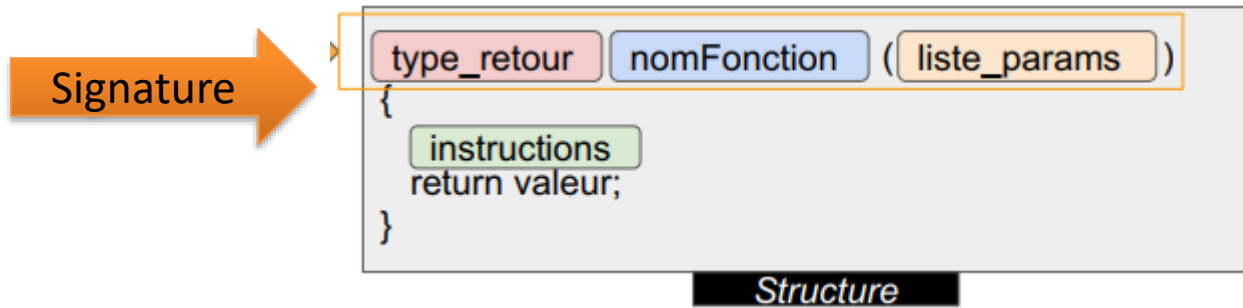
Un ensemble de paramètres en entrée qui permettent de varier les instructions de la fonction

Un résultat de sortie

Introduction

- Une fonction est un sous-programme qui accomplit une tâche spécifique.
- Elle possède une structure analogue à celle de la fonction main.
- L'utilisation de fonctions présente plusieurs avantages :
 - Rendre le programme plus facile à écrire et beaucoup plus facile à lire.
 - Eviter d'écrire plusieurs fois la même séquence de code.
 - La mise au point est plus simple car les différentes fonctions peuvent être testées séparément.
 - Elles peuvent être réutilisées par d'autres programmes.

Les fonctions



```
int main()
{
    return 0;
}
```

Exemple 1

```
void maFonction()
{
    ...
}
```

Exemple 2

```
void afficherAge(int age)
{
    printf("tu a %d ans\n", age);
    return;
}
```

Exemple 3

```
int somme(int var1, int var2)
{
    int total= var2 + var1;
    return total;
}
```

Exemple 4

Signature comme une carte d'identité qui va nous donner comment ca marche la fonction

Règles de nommage des fonctions

<u>Règles</u>
Pas de nombre en début de nom
Pas de caractères spéciaux à l'exception de _
Pas de mots réservés en c (ex type de variable)
Pas d'accent (à, é, è , ...)
Pas d'espace
Nom explicite (très important pour la lisibilité)
Minuscule sauf 1ere lettre mots séparateurs

<u>Exemples</u>
2fonction
nom-de-fonction
for
début
nom de fonction
nom_de_fonction
nomDeFonction

Les fonctions

Passer sur code::Blocks

Créer une fonction qui s appelle ditHello

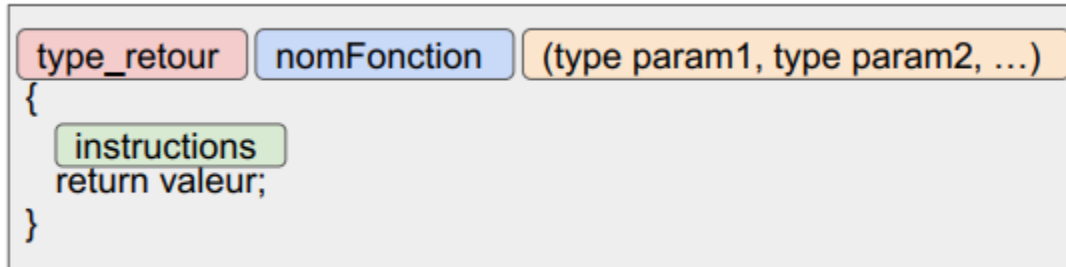
```
#include <stdio.h>
#include <stdlib.h>
void ditHello()
{
    printf("Hello world!\n");
    printf("cours du langage C \n");
}

int main()
{
    ditHello();
    ditHello();
    ditHello();

    return 0;
}
```

```
Hello world!
cours du langage C
Hello world!
cours du langage C
Hello world!
cours du langage C
```

Paramètres de fonction



```
void somme(int var1, int var2)  
{  
    int somme = var1+var2;  
    printf("%d+%d=%d", var1, var2, somme);  
}  
  
int main()  
{  
    int ma_variable = 2;  
    somme(3,1);  
    somme(3, ma_variable);  
}
```

3+1=4
3+2=5

Paramètres de fonction

Passer sur code::Blocks

Créer une fonction qui va afficher des infos sur une personnage vidéo

```
void ditHello()
{
    printf("Hello world!\n");
    printf("cours du langage C \n");
}
void infoPerso(int xp, int lvl)
{
    printf("\txp= %d\n",xp);
    printf("\tlvl= %d\n",lvl);
}
int main()
{
    ditHello();
    printf("Mage :");
    infoPerso(1250,10);

    return 0;
}
```

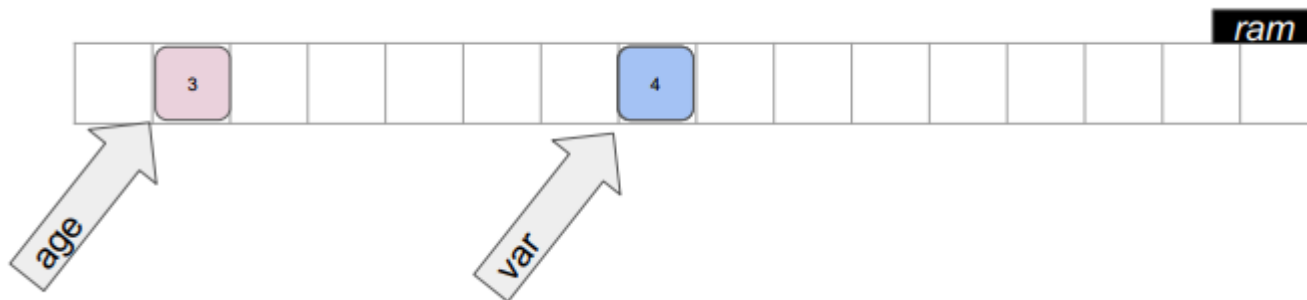
```
Hello world!
cours du langage C
Mage : xp= 1250
      lvl= 10
```

Passage par copie
Passage par valeur

Passage de paramètre par copie

```
int main()  
{  
    int age= 3;  
    anniversaire(age);  
}
```

```
void anniversaire(int var)  
{  
    var++;  
    printf("%d", var);  
}
```

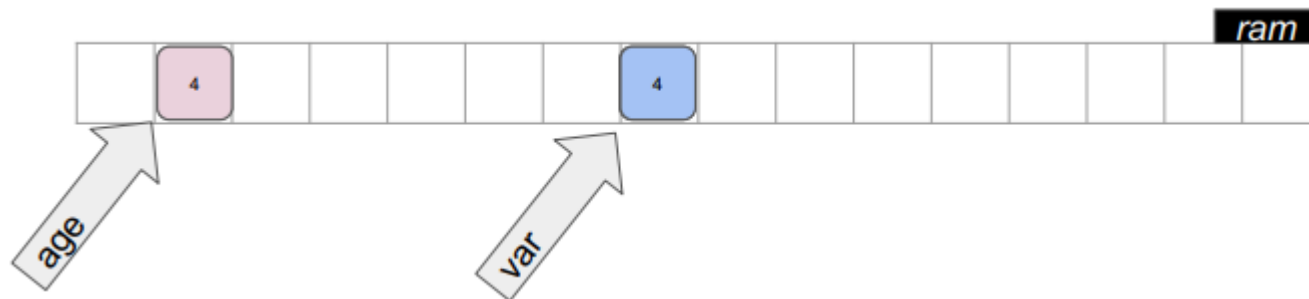


Passage par copie ou passage par valeur

Récupération du retour de fonction

```
int main()  
{  
    int age= 3;  
    age = anniversaire(age);  
}
```

```
int anniversaire(int var)  
{  
    var++;  
    return var;  
}
```



Passage de paramètre

Passer sur code::Blocks

Créer une fonction qui va afficher des infos sur un personnage vidéo

```
void ditHello()
{
    printf("Hello world!\n");
    printf("cours du langage C \n");
}
void infoPerso(int xp, int lvl)
{
    xp*=2;
    printf("\txp= %d\n",xp);
    printf("\tlvl= %d\n",lvl);
}
int main()
{
    int xp_mage=1250;
    ditHello();
    printf("Mage :");
    infoPerso(xp_mage,10);
    printf("\txp =%d\n",xp_mage);

    return 0;
}
```

```
Hello world!
cours du langage C
Mage : xp= 2500
      lvl= 10
      xp =1250
```

Les returns

```
void balrog()  
{  
    return;  
    printf("Vous ne passerez pas!");  
}  
  
int main()  
{  
    printf("Salut \n");  
    balrog();  
    return 0;  
}
```

Salut

Exemple

Les returns

Passer sur code::Blocks

Fonction qui calcule puissance (2^3)

```
#include <stdio.h>
#include <stdlib.h>
int puissance (int base, int exposant )
{
    int resultat ;
    int i;
    if (base==0 && exposant ==0)
    {
        return 0;
    }
    if (base==0 && exposant !=0)
    {
        return 0;
    }
    if (base!=0 && exposant ==0)
    {
        return 1;
    }
    resultat=1;
    for(i=0; i< exposant; i++)
    {
        resultat=resultat*base;
    }
    return resultat;
}
```

```
0^3=0
2^0=1
2^3=8
```

```
int main()
{

    printf("0^3=%d\n", puissance(0,3));
    printf("2^0=%d\n", puissance(2,0));
    printf("2^3=%d\n", puissance(2,3));
    return 0;
}
```

Exercice : lanceur de dés

1- Créer une fonction De, qui prend en entrée 1 entier qui correspond au nombre de faces d'un dé et qui retourne aléatoirement un numéro de face.

Exemple de(6) correspond à un lancé de dé de 6, donc un retour entre 1 et 6.

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
int de(int);
int main()
{
    // initialisation de la génération aléatoire
    srand(time(NULL));
    printf("on lance un De de %d :%d",6,de(6));
    return 0;
}
int de(int nb_faces)
{
    int resultat = (rand()% nb_faces)+1;
    return resultat;
}
```

1

on lance un De de 6 :6

Exercice : lanceur de dés

2- Créer une fonction Des, qui prend en entrée 2 entiers - Nombre de dés à lancer. - Nombre de faces par dé.

Cette fonction retourne alors la somme des résultats de chaque dé.

```
#include <stdio.h>#include <stdio.h>
#include <stdlib.h>
#include <time.h>
int de(int);
int des(int, int);
int main()
{ // initialisation de la génération aléatoire
  srand(time(NULL));

  printf("on lance %d de Des de %d :%d\n",3,6,des(3,6));
  return 0;
}
int de(int nb_faces)
{
  int resultat = (rand()% nb_faces)+1;
  return resultat;
}
int des (int nb_des,int nb_faces)
{
  int i,No_face;
  int total=0;
  for(i=1;i<=nb_des;i++)
  {

    No_face=de(nb_faces);

    printf("\n la face du de numero %d = %d\n", i, No_face);
```

2

la face du de numero 1 = 2

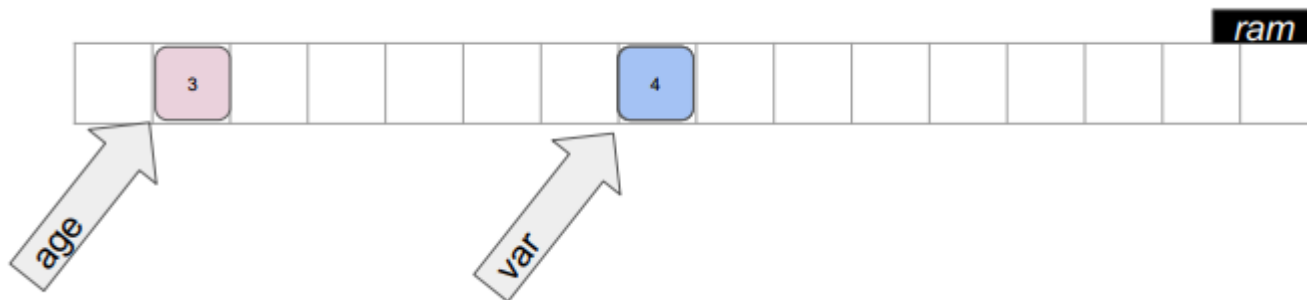
la face du de numero 2 = 1

la face du de numero 3 = 6
on lance 3 de Des de 6 :9

Passage de paramètre par copie

```
int main()  
{  
    int age= 3;  
    anniversaire(age);  
}
```

```
void anniversaire(int var)  
{  
    var++;  
    printf("%d", var);  
}
```

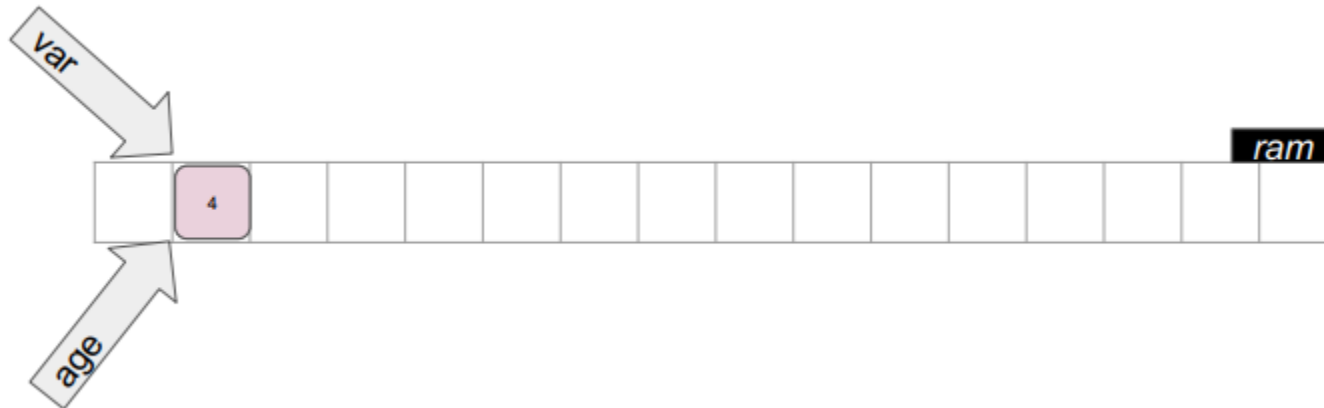


Passage par copie ou passage par valeur

Passage des paramètres par pointeur

```
int main()  
{  
    int age= 3;  
    anniversaire(&age);  
}
```

```
void anniversaire(int* var)  
{  
    *var++;  
    printf("%d", *var);  
}
```



Passage des paramètres par pointeur

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
```

```
void inverser(int nb1,int nb2)
{
    int tmp =nb2;
    nb2= nb1;
    nb1= tmp;
}
```

```
int main()
{
    int nombre_1=12;
    int nombre_2=89;
    printf("nombre_1 =%d\n",nombre_1);
    printf("nombre_2 =%d\n",nombre_2);
    inverser(nombre_1,nombre_2);
    printf("inverser :\n");
    printf("nombre_1 =%d\n",nombre_1);
    printf("nombre_2 =%d\n",nombre_2);

    return 0;
}
```

```
nombre_1 =12
nombre_2 =89
inverser :
nombre_1 =12
nombre_2 =89
```

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
```

```
int inverser(int* nb1,int* nb2)
{
    int tmp ;
    if(nb1==NULL || nb2==NULL)
    {
        return -1;//sortir de la fonction
    }
    tmp =*nb1;
    *nb1= *nb2;
    *nb2= tmp;
    return 0;
}
```

```
int main()
{
    int nombre_1=12;
    int nombre_2=89;
    printf("nombre_1 =%d\n",nombre_1);
    printf("nombre_2 =%d\n",nombre_2);
    if(inverser(&nombre_1,&nombre_2) == 0 )
    {
        printf("inverser !\n");
    }
    printf("nombre_1 = %d\n", nombre_1);
    printf("nombre_2 = %d\n", nombre_2);
    return 0;
}
```

```
nombre_1 =12
nombre_2 =89
inverser !
nombre_1 = 89
nombre_2 = 12
```


Seance suivante

Test

- Donner l'affichage du programme suivant

```
#include <stdio.h>
#include <stdlib.h>
int main()
{
    int i=10;
    while(i>0)
    {
        i=i-4;
        printf("%d\t", i);
    }
    return 0;
}
```

Le passage de tableau

Sur code::blocks, trouver le nombre max sur un tableau a une dimension

```
#include <stdio.h>
#include <stdlib.h>

int max1D(int tab[], int taille)
{
    int valeur_max = tab[0];
    int i;
    for(i=1;i<taille;i++)
    {
        if (valeur_max<tab[i])
            valeur_max=tab[i];
    }
    return valeur_max;
}

int main()
{
    int tab1D[5]={22,15,78,1,-6};
    int nombre_max_1D = max1D(tab1D,5);
    printf("le nombre le plus grand du tab1D est %d \n",nombre_max_1D);

    return 0;
}
```

le nombre le plus grand du tab1D est 78

Le passage de tableau

Sur code::blocks, trouver le nombre max sur un tableau a deux dimensions

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
int max1D(int tab[], int taille)
{
    int valeur_max =0;
    int i;
    for(i=0;i<taille;i++)
    {
        if(tab[i]>valeur_max)
            valeur_max=tab[i];
    }
    return valeur_max;
}
```

```
int max2D(int tab[][5], int nb_ligne,int nb_colonne)
{
    int valeur_max =0;
    int i,j;
    for(i=0;i<nb_ligne;i++)
    {
        for(j=0;j<nb_colonne;j++)
        if(tab[i][j]>valeur_max)
        {
            valeur_max=tab[i][j];
        }
    }
    return valeur_max;
}
```

```
int main()
{
    int tab1D[5]={22,50,14,17,5};
    int tab2D[2][5]={{22,50,2,6,14},{2,5,2,6,94}};
    int nombre_max_1d=max1D(tab1D,5);
    int nombre_max_2d=max2D(tab2D,2,5);
    printf("le nombre le plus grand du tab1D est
%d\n",nombre_max_1d);
    printf("le nombre le plus grand du tab2D est
%d\n",nombre_max_2d);

    return 0;
}
```

le nombre le plus grand du tab1D est 50
le nombre le plus grand du tab2D est 94

Fonction et pointeur de tableau

```
#include <stdio.h>
#include <stdlib.h>

int max2D(int* tab, int nb_ligne, int nb_colonne)
{
    int valeur_max = *(tab);
    int i,j;
    int *p_ligne = NULL;
    for(i=0;i<nb_ligne;i++)
    {
        p_ligne=tab+(i*nb_colonne);

        for(j=0;j<nb_colonne;j++)

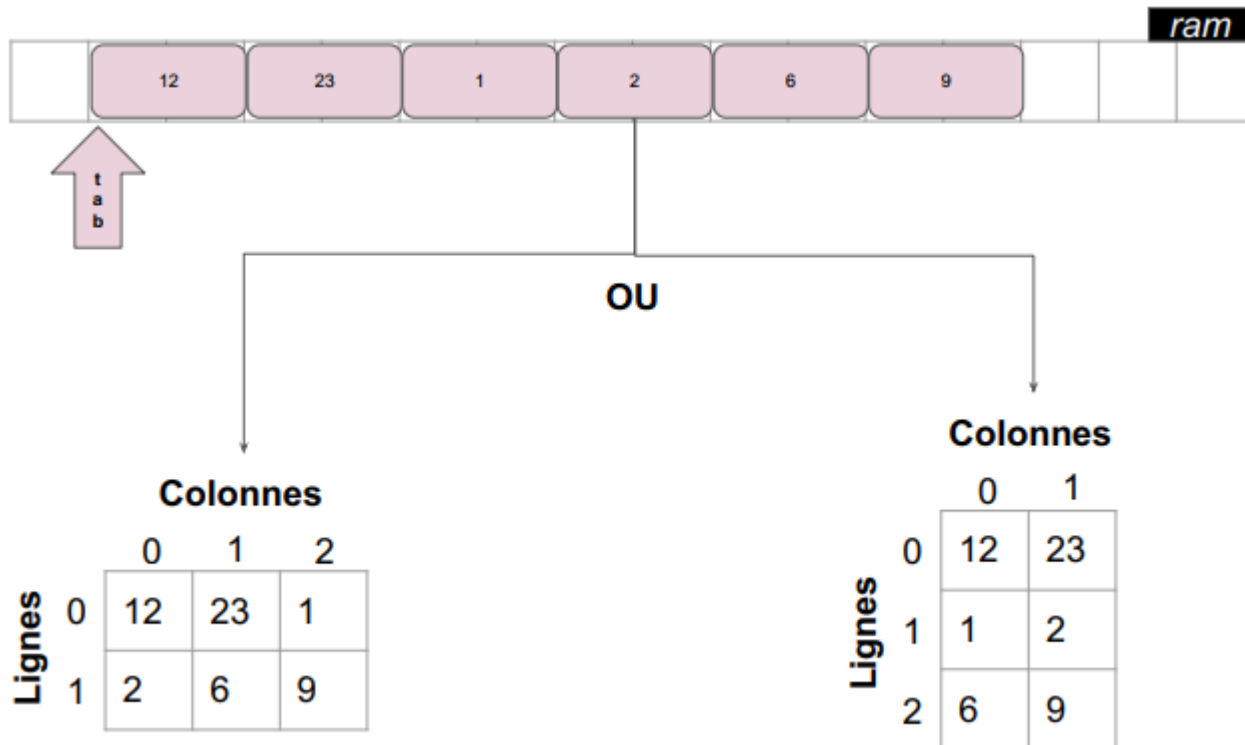
            if (valeur_max<*(p_ligne+j))
                valeur_max=*(p_ligne+j);
    }
    return valeur_max;
}

int main()
{
    int tab2D[2][5]={22,15,78,1,-6},{14,18,98,11,46}};
    int nombre_max_2D = max2D((int*)tab2D,2,5);

    printf("le nombre le plus grand du tab2D est %d \n",nombre_max_2D);
    return 0;
}
```

le nombre le plus grand du tab2D est 98

Structure d'un tableau



Prototype des fonctions

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
```

```
int max1Dpointeur(int* tab, int taille); //int max1Dpointeur(int*, int );
int max2DPointeur(int* tab, int nb_ligne, int nb_colonne); //int
max2DPointeur(int* , int ,int);
```

```
int main()
{
    int tab1D[5]={22,50,14,17,5};
    int tab2D[2][5]={{22,50,2,6,14},{2,5,2,6,94}};
    int nombre_max_1d=max1Dpointeur(tab1D,5);
    int nombre_max_2d=max2DPointeur((int*)tab2D,2,5);
    printf("le nombre le plus grand du tab1D est %d\n",nombre_max_1d);
    printf("le nombre le plus grand du tab2D est %d\n",nombre_max_2d);

    return 0;
}
```

```
int max1Dpointeur(int* tab, int taille)
{
    int valeur_max =0;
    int i;
    for(i=0;i<taille;i++)
    {
        if(*(tab+i)>valeur_max)
            valeur_max=*(tab+i);
    }
    return valeur_max;
}
```

le nombre le plus grand du tab1D est 50
le nombre le plus grand du tab2D est 94

```
int max2DPointeur(int* tab, int nb_ligne, int nb_colonne)
{
    int valeur_max =0;
    int i,j;
    int *p_ligne=0;
    for(i=0;i<nb_ligne;i++)
    {
        p_ligne=tab+(i*nb_colonne);
        for(j=0;j<nb_colonne;j++)

            if(*(p_ligne+j)>valeur_max)
            {
                valeur_max= *(p_ligne+j);
            }
    }
    return valeur_max;
}
```

Prototype des fonctions

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
int max1D(int [], int );
int max2D(int [][][5], int ,int );
int main()
{
    int tab1D[5]={22,50,14,17,5};
    int tab2D[2][5]={{22,50,2,6,14},{2,5,2,6,94}};
    int nombre_max_1d=max1D(tab1D,5);
    int nombre_max_2d=max2D(tab2D,2,5);
    printf("le nombre le plus grand du tab1D est
%d\n",nombre_max_1d);
    printf("le nombre le plus grand du tab2D est
%d\n",nombre_max_2d);

    return 0;
}
```

```
int max1D(int tab[], int taille)
{
    int valeur_max =0;
    int i;
    for(i=0;i<taille;i++)
    {
        if(tab[i]>valeur_max)
            valeur_max=tab[i];
    }
    return valeur_max;
}
```

```
int max2D(int tab[][][5], int nb_ligne,int nb_colonne)
{
    int valeur_max =0;
    int i,j;
    for(i=0;i<nb_ligne;i++)
    {
        for(j=0;j<nb_colonne;j++)
        {
            if(tab[i][j]>valeur_max)
            {
                valeur_max=tab[i][j];
            }
        }
    }
    return valeur_max;
}
```

le nombre le plus grand du tab1D est 50
le nombre le plus grand du tab2D est 94

Quizz kahoot

Devoir a rendre : jeu de Morpion

Nous allons créer un petit jeu de type Morpion. L'objectif est d'aligner 3 symboles identiques dans une grille de 3X3. Le jeu se joue à deux. Chaque joueur possède un symbole parmi 'X' et 'O'. A tour de rôle, ils dessinent dans un cas libre de la grille leur symbole respectif. Celui qui aligne 3 de ses symboles (horizontal, vertical ou diagonal), remporte la victoire. En cas de grille pleine sans alignement de 3 symboles identiques, c'est un match nul !

```
...
|X|O|X|
|X|O|O|
|. |X|O|
Joueur X ou voulez-vous jouer (x y)? 1 1
Case deja occupee!
Joueur X ou voulez-vous jouer (x y)? 2 0
Victoire du joueur X
|X|O|X|
|X|O|O|
|X|X|O|
```

FIN

Seance de revision

- Travailler deux examens
- A envoyer au delegué